

**Stacks 패키지 레지스트리를 위한
Clarity 스마트 컨트랙트 순위 산정:
ERC-20 allowance를 대신하는 포스트컨디션**

권태홍 (Taehong Kwon)

남아프리카대학교 (University of South Africa)

원고 초안, 2026년 10월 7일

초록

Stacks에 배포된 Clarity 스마트 컨트랙트는 모두 사람이 읽을 수 있는 소스 형태로 체인에 저장된다. 그런데도 개발자가 이 컨트랙트를 찾아 다시 쓸 수 있는, npm이나 pkg.go.dev 같은 레지스트리는 없다. 이런 레지스트리에는 복제본과 스팸을 사람들이 실제로 의존하는 컨트랙트보다 아래에 두고, 공격자가 가짜 신원으로 싸게 살 수 없는 순위가 필요하다. 선행 연구는 이더리움 지갑의 순위를 ERC-20 전송 위의 PageRank(AWP)나 ERC-20 allowance 위의 PageRank(EndorseRank)로 매겼다. Stacks에는 allowance가 없다. SIP-010 토큰 표준에는 approve도 transferFrom도 없다. 사용자는 컨트랙트를 직접 호출해 자기 토큰을 옮기게 하고, 서명한 트랜잭션에 붙인 포스트컨디션으로 자기 계정에서 나갈 수 있는 양을 제한한다. Clarity 4부터는 컨트랙트도 as-contract?와 restrict-assets?의 자산 한도로 자기 자산의 유출을 제한한다.

이 논문은 allowance 관계를 이 메커니즘들에 대응시키고, 노드가 컨트랙트뿐인 순위 PC-EndorseRank를 정의한다. 포스트컨디션 승인을 EndorseRank처럼 지갑·컨트랙트·자산마다 최신 상태 하나로 줄이고, 이것으로 각 지갑이 보증을 여러 컨트랙트에 어떻게 나누는지를 정한다. 지갑이 낸 수수료는 그 보증을 얼마나 무겁게 셀지를 정한다. 컨트랙트 코드의 위임 참조와 의존 참조는 점수를 컨트랙트에서 컨트랙트로 옮긴다. 균등 재시작을 쓰면 컨트랙트 점수 가운데 지갑에서 오는 부분이 지갑 수를 나누어 센 값이 되어 시빌 지갑을 막지 못한다는 것을 보이고, 공격자 컨트랙트 묶음이 모을 수 있는 점수의 상한을 그 묶음의 재시작 몫과 정직한 컨트랙트에서 홀러드는 점수로 나타낸다. 합성 예제로 두 결과를 보인다. 이어서 레지스트리와 명령줄 클라이언트 cpm을 설명한다. cpm은 배포된 컨트랙트를 참조하는 일과 그 소스를 복제하는 일을 구분한다. 끝으로 라벨 기간이 열리기 전에 고정한 Stacks 메인넷 평가 계획을 제시한다. 이 평가는 공격자가 컨트랙트를 끌어올리는 데 드는 비용과, 순위가 이후의 채택과 얼마나 맞는지를 개수 기반, 의존성 PageRank, AWP, EndorseRank 기준선과 비교한다.

주제어: Stacks; Clarity; 스마트 컨트랙트; 패키지 레지스트리; PageRank; 시빌 공격 저항; 포스트컨디션; 소프트웨어 공급망

1 서론

Stacks의 스마트 컨트랙트 언어인 Clarity는 컴파일하지 않고 해석하는 언어이며, 배포된 컨트랙트는 노드가 실행하는 소스 텍스트 그대로 체인에 저장된다 [1, 2]. 그래서 누구나 블록 탐색기나 Stacks Blockchain API로 Stacks 메인넷의 어떤 컨트랙트든 읽을 수 있고 [3], 배포된 모든 컨트랙트의 소스를 GitHub에 옮겨 두는 공개 저장소도 있다 [4]. 이와 달리 이더리움에서는 개발자가 소스를 공개하고 검증을 받은 컨트랙트만 소스를 볼 수 있다. 기존 Clarity 컨트랙트 위에 무언가를 만들려는 개발자에게는 두 가지 길이 있다. 배포된 컨트랙트를 그 `principal`을 지정한 `contract-call?`로 호출하거나, 소스를 복제해 자기 주소로 새 인스턴스를 배포하는 것이다. 표준 개발 도구인 Clarinet은 첫 번째 길을 이미 지원한다. `clarinet requirements add`는 메인넷 컨트랙트를 프로젝트로 내려받아 로컬 테스트에서 호출할 수 있게 한다 [5].

Stacks에 없는 것은 npm, crates.io, pkg.go.dev가 코드 위에 얹어 주는 층이다. 검색, 문서, 버전 계보, 게시자 신원, 그리고 다른 사람들이 의존하는 컨트랙트를 복제본·테스트·스팸보다 위에 두는 순서가 여기에 든다. 패키지 레지스트리는 보통 인기 신호로 검색 결과를 정렬한다. 예컨대 Go 패키지 사이트는 모듈의 텍스트 관련도에 그 모듈을 가져오는 패키지 수의 로그를 곱한다 [6]. 이런 개수는 부풀리기 쉽다. 2024년 4월 Sonatype은 tea 프로토콜의 보상을 받으려고 여러 계정에서 게시한 npm 패키지 약 14,000개를 보고했다 [7]. tea 프로토콜은 의존성 그래프에서 차지하는 위치로 오픈소스 패키지에 점수를 매긴다 [8]. 2025년 11월까지 Amazon Inspector는 같은 캠페인에 엮인 npm 패키지 150,000개 이상을 찾아냈다. 이 패키지들은 서로를 의존성으로 선언해서 하나를 설치하면 나머지가 모두 따라 들어왔다 [9]. 개발자가 어떤 컨트랙트를 설치할지를 순위가 정하는 Stacks 레지스트리도 같은 행동을 부를 것이다. 잘못된 추천의 피해는 더 크다. 참조한 컨트랙트가 실제 자산을 보관하거나 옮길 수 있기 때문이다.

선행 연구는 PageRank로 이더리움 지갑의 순위를 매겼다. Adaptive Weighted PageRank(AWP)는 ERC-20 전송을 금액과 경과 시간에 대한 로지스틱 감쇠로 가중한다 [10]. EndorseRank는 소유자-지출자 간선마다 둘 사이의 최신 ERC-20 allowance로 가중한다. 승인은 동기가 여러 가지일 수 있는 지금이 아니라 지출 권한을 명시적으로 내준 것이라는 관점이다 [11]. 이 논문은 같은 생각으로 Clarity 컨트랙트의 순위를 매길 수 있는지 묻는다. 두 가지 문제가 있다.

첫째, Stacks에는 allowance가 없다. SIP-010 대체 가능 토큰 trait는 transfer를 정의하지만 approve, allowance, transferFrom은 없다 [12]. 사용자는 컨트랙트를 직접 호출해 그 컨트랙트가 자기 토큰을 옮기게 한다. 컨트랙트는 사용자 자신의 트랜잭션 안에서 움직이고, 사용자는 포스트컨디션으로 자기 계정에서 나갈 수 있는 양을 제한한다. 프로토콜은 실행이 끝난 뒤 포스트컨디션을 검사한다 [13]. Clarity 4부터는 컨트랙트도 자기가 통제하지 않는 코드를 실행하는 동안 as-contract?와 restrict-assets?의 자산 한도로 자기 자산의 유출을 제한할 수 있다 [14]. 이 메커니즘들은 함께 allowance가 기록하는 내용의 대부분을 담지만, 저마다 하나의 트랜잭션이나 하나의 코드 블록에 묶여 있을 뿐 계속 남는 잔액으로 저장되지는 않는다. 4절에서 이 가운데 무엇이 어떻게 allowance 간선의 역할을 할 수 있는지 따진다.

둘째, PageRank만으로는 시빌(Sybil) 공격을 막지 못한다. Cheng과 Friedman은 노드의 신원을 대칭적으로 다루는 평판 함수는 시빌 공격에 안전할 수 없음을 증명했고 [15], 링크 팜은 대상 페이지 주위에 그 페이지를 가리키는 페이지들을 만들어 PageRank를 올린다 [16]. 저항력은 비대칭에서 나온다. TrustRank와 SybilRank는 신뢰하는 노드에서만 걷기를 시작하거나 다시 시작한다 [17, 18]. 5절에서는 지갑이 자신이 승인한 컨트랙트를 가리키는 그래프에서 균등 재시작 PageRank가 모든 지갑에 같은 무게를 준다는 것을 보인다. 그 결과 컨트랙트 점수 가운데 지갑에서 오는 부분은 그 컨트랙트를 쓰는 지갑 수를 나누어 센 값이 된다. 재시작을 바꾸지 않고 EndorseRank를 Stacks로 옮기면, 호출자 수를 세는 것만큼이나 조작하기 쉬운 순위가 된다.

이 논문의 기여는 네 가지이다.

- **allowance 관계의 Stacks 대응:** ERC-20 승인을 포스트컨디션, Clarity 4 자산 한도, 옛 as-contract, 확장 맵, 정적 참조와 비교하고, 지갑→컨트랙트 보증에는 포스트컨디션을, 컨트랙트→컨트랙트 간선에는 코드 수준의 위임과 의존을 고른다. 어떤 포스트컨디션을 승인으로 셀지 규칙을 정한다. AWP는 대체할 필요가 없다. Stacks는 자산 전송을 기록하기 때문이다.
- **PC-EndorseRank:** 컨트랙트만을 노드로 두고 지갑은 재시작 분포로만 들어오는 순위이다. EndorseRank의 최신 상태 규칙을 유지하고, 각 지갑을 그 지갑이 낸 수수료로 가중하며, 점수를 위임 간선과 의존 간선을 따라 전파한다. 세 개의 명제로 이 순위와, 균등 재시작으로 Stacks에 옮긴 EndorseRank가 시빌 공격에 어떻게 반응하는지 기술하고, 합성 예제로 보인다.

- 레지스트리와 명령줄 클라이언트: Clarity Registry와 cpm은 배포된 컨트랙트를 참조하는 일(cpm add)과 소스를 복제하는 일(cpm create, cpm fork)을 구분한다. 게시자 스코프는 배포자 주소와 BNS 이름에서 가져오고, 버전은 게시자가 선언한 계보로 다루며, 클론은 정규화한 소스 해시와 contract-hash?로 묶고, 코드 해시를 잠금 파일에 고정한다.
- 미리 고정한 평가 계획: Stacks 메인넷에서 PC-EndorseRank를 단순 개수, 의존성 PageRank, AWP, 균등 재시작 EndorseRank와 비교한다. 공격자가 순위 상위에 오르는 데 드는 비용, 이후 기간의 채택과의 일치, 검색 결과 중 클론의 비율을 잴다. 단순 개수를 넣는 까닭은 우리의 ERC-20 연구에서 떼어 둔(held-out) 승인과 더 잘 맞는 쪽이 두 PageRank 변형이 아니라 단순 개수였기 때문이다 [11].

2절은 Stacks와 Clarity, 자산 이동을 통제하는 메커니즘, PageRank와 레지스트리의 배경을 설명한다. 3절은 관련 연구를 검토한다. 4절은 allowance를 Stacks에 대응시키고, 5절은 PC-EndorseRank를 정의하며, 6절은 레지스트리와 cpm을 설명한다. 7절은 평가 계획을, 8절은 한계를 다루고, 9절에서 맺는다.

2 배경

2.1 Stacks와 Clarity

Stacks는 비트코인에 정산하는 스마트 컨트랙트 블록체인이다 [19]. 컨트랙트는 Clarity로 작성한다. Clarity에는 재귀와 무한 반복이 없어서 호출 비용의 상한을 실행 전에 정할 수 있다 [1]. Clarity는 컴파일하지 않는다. 노드는 소스 텍스트를 저장하고 해석하며, Stacks Blockchain API는 배포된 어느 컨트랙트에 대해서든 그 텍스트와 컨트랙트 인터페이스 (ABI)를 돌려준다 [3].

컨트랙트는 SP...ABC.amm-pool 꼴의 컨트랙트 principal로 식별한다. 점 앞의 주소는 그 컨트랙트를 배포한 standard principal이므로, 식별자 자체에 코드를 게시한 사람이 기록된다. 컨트랙트는 contract-call?로 다른 컨트랙트를 호출한다. 정적 디스패치에서는 피호출자를 principal 리터럴로 쓰고, 동적 디스패치에서는 피호출자가 trait 타입의 함수 인자여서 호출하는 쪽이 실행 시점에 구체적인 컨트랙트를 고른다 [1]. trait는 define-

trait로 선언한다. 컨트랙트는 impl-trait로 어떤 trait를 따른다고 밝히고, use-trait로 trait를 가져와 인자 타입으로 쓴다. 두 구문 모두 그 trait를 정의한 컨트랙트를 가리킨다. Clarity 5는 상수도 contract-call?의 대상으로 받는다 [20]. 따라서 정적 디스패치는 소스에서 읽어 낼 수 있는 호출 그래프를 만들고, 동적 디스패치는 트랜잭션이 피호출자를 넘겨줄 때까지 그 자리를 비워 둔다.

토큰 표준은 trait이다. SIP-009는 대체 불가능 토큰을 [21], SIP-010은 대체 가능 토큰을 [12], SIP-013은 준대체 가능 토큰을 정의한다 [22]. Stacks Blockchain API는 ABI가 주어진 trait와 맞는 컨트랙트를 나열할 수 있어서 [3], 레지스트리는 컨트랙트를 파싱하지 않고도 표준을 따르는 컨트랙트에 표시를 붙일 수 있다.

2.2 Stacks가 자산 이동을 승인하는 방식

SIP-010에는 **allowance**가 없다. SIP-010 trait는 금액, 보내는 쪽, 받는 쪽, 선택적 메모를 받는 transfer 함수와, 이름·심볼·소수 자릿수·잔액·공급량·토큰 URI를 돌려주는 읽기 전용 함수들을 선언한다 [12]. 한 principal이 다른 principal의 토큰을 쓰게 하는 함수는 없다. 흔한 구현은 보내는 쪽이 tx-sender일 때만 전송을 받아들인다. Clarity에서 tx-sender는 컨트랙트가 맥락을 자기 자신으로 바꾸지 않는 한 중첩 호출을 거처도 트랜잭션에 서명한 계정으로 남고, contract-caller는 바로 앞의 호출자이다 [1]. 그래서 사용자가 호출한 탈중앙 거래소는 미리 승인을 받지 않고도 그 호출 안에서 사용자의 토큰을 옮길 수 있다. 사용자를 지키는 것은 포스트컨디션 메커니즘이다.

포스트컨디션. Stacks 트랜잭션은 포스트컨디션 목록과 포스트컨디션 모드를 담고, 둘 다 서명 대상에 포함된다 [13]. 각 포스트컨디션은 principal(트랜잭션 발신자, standard principal, 컨트랙트 principal 중 하나), 자산(STX, 대체 가능 토큰, 특정 대체 불가능 토큰 중 하나), 조건을 지정한다. STX와 대체 가능 토큰의 조건은 그 principal이 트랜잭션 동안 보내는 양을 정해 둔 양과 $=$, $>$, $\geq$, $<$, $\leq$ 로 비교한다. 대체 불가능 토큰의 조건은 그 principal이 해당 토큰을 보낼지 보내지 않을지를 정한다. 실행이 끝나면 노드가 모든 포스트컨디션을 검사한다. 하나라도 어긋나면 트랜잭션은 중단되고 그 효과는 되돌려진다. 다만 트랜잭션은 블록에 남고 수수료도 부과된다. 모드는 어느 포스트컨디션에도 나오지 않는 전송을 어떻게 다룰지를 정한다. Allow 모드에서는 그런 전송을 허용하고, Deny 모드에서는 트랜잭션을 중단한다 [13]. Stacks.js는 따로 지정하지 않으면 Deny 모드로 트랜잭션을 만든다 [23]. Clarity 5와 함께 Epoch 3.4(비트코인 블록 943,333, 2026년 4월)에 적용된 SIP-040은 세

번째 모드인 *Originator*를 더했다. 이 모드는 트랜잭션 발신자의 자산에는 Deny 규칙을, 다른 모든 principal에는 Allow 규칙을 적용한다. SIP-040은 대체 불가능 토큰에 *may send* 조건도 더했다 [24, 20]. 도입 동기는 DeFi 호출이 사용자가 미리 알 수 없는 방식으로 컨트랙트 사이에 자산을 옮겨서 사용자를 Allow 모드로 내몰았다는 데 있다.

컨트랙트의 자산 한도. 2025년 11월 비트코인 블록 923,222에서 활성화된 Clarity 4(SIP-033)는 컨트랙트가 자기가 통제하지 않는 코드를 실행하는 동안 자산 유출을 제한할 수 있게 한다 [14]. *restrict-assets?*는 principal 하나, 한도 목록, 본문을 받아 본문을 실행한 뒤 그 principal의 유출을 한도와 비교한다. *as-contract?*는 tx-sender와 contract-caller를 컨트랙트 자신으로 바꾸어 본문을 실행하고, 컨트랙트 자신의 유출을 검사한다. 한도 표현식은 with-stx, with-ft, with-nft, with-stacking이며 각각 자산 하나와 그 한도를 지정한다. *as-contract?*에서만 쓸 수 있는 with-all-assets-unsafe는 모든 한도를 없앤다. 2026년 7월 비트코인 블록 960,230에서 Epoch 4.0과 함께 적용된 Clarity 6은 with-stacking의 이름을 with-staking으로 바꾸고, PoX 컨트랙트 호출을 위한 with-pox를 더했다. 새로운 종류의 트랜잭션 포스트컨디션은 더하지 않았다 [25]. 유출이 한도를 넘으면 본문의 효과는 되돌려지고, 표현식은 어긴 한도의 순번을 오류로 돌려준다. 한도가 없는 자산이 움직이면 (err u128)을 돌려준다. Clarity 4는 아무 한도 없이 맥락만 바꾸던 옛 *as-contract*를 없앴다. 코드 1에 이 패턴을 보였다. SIP-033은 *contract-hash?*도 더했다. 이 함수는 컨트랙트 코드 본문의 SHA-512/256 해시를 돌려주어, 한 컨트랙트가 다른 컨트랙트가 알려진 템플릿을 따르는지 확인할 수 있게 한다. 키워드 *current-contract*도 이때 더해졌다.

코드 1: Clarity 4의 자산 한도. 볼트 소유자만 지급을 시작할 수 있고, 본문이 실행되는 동안 볼트에서 나갈 수 있는 토큰은 최대 amount이다.

```
(define-constant OWNER tx-sender)
(define-public (pay-out (amount uint) (recipient principal))
  (begin
    (asserts! (is-eq contract-caller OWNER) (err u401))
    (as-contract? ((with-ft .token "tok" amount)
      (try! (contract-call? .token transfer amount current-contract
        recipient none))))))
```

컨트랙트 사이의 지속적인 승인. 일부 프로토콜은 신뢰하는 컨트랙트의 목록을 명시적으로 둔다. 여러 Stacks DAO가 쓰는 ExecutorDAO 패턴에서는 핵심 컨트랙트가 활성화된 확장의 맵을 두고, 거버넌스가 set-extension을 호출해 확장을 켜거나 끈다. 켜진 확장은 contract-caller 검사를 거쳐 DAO의 송신 맥락에서 행동할 수 있다 [26]. 이런 맵은

Stacks에서 지속적인 allowance에 가장 가까운 것이다. 명시적인 트랜잭션이 설정하고, 바뀔 때까지 효력이 남는다.

2.3 패키지 레지스트리와 인기 신호

패키지 레지스트리는 게시된 코드를 색인하고 텍스트 관련도와 인기를 섞어 검색 결과의 순위를 매긴다. Go 패키지 사이트는 질의와 맞는 모듈마다 다음 점수를 계산한다.

$$\text{score} = \text{ts_rank}(q, m) \cdot (\ln(e + \text{imported_by}(m)))^\alpha \cdot \rho_{\text{redist}}(m) \cdot \rho_{\text{mod}}(m) \quad (1)$$

여기서 ts_rank 는 PostgreSQL의 전문 검색 순위, imported_by 는 그 모듈을 가져오는 패키지 수이고, 마지막 두 인자는 재배포할 수 없거나 `go.mod` 파일이 없는 모듈에 벌점을 준다 [6]. 로그 때문에 가져오는 패키지 수가 두 배가 되면 그 인자는 두 배가 되지 않고, 그 로그 값이 약 $\ln 2$ 만큼 커질 뿐이다. npm, Cargo, CRAN 등 여러 생태계의 의존성 네트워크는 그래프로 연구되어 왔고 [27, 28], npm 의존성 그래프 위의 PageRank로 쇠퇴하는 패키지를 찾는 연구도 있다 [29].

tea 프로토콜은 의존성 그래프에서 패키지가 차지하는 자리를 점수로 매긴 teaRank에 따라 보상을 지급했다 [8]. 1절에서 본 사건들은 이런 점수에 돈이 걸리면 무슨 일이 생기는지 보여 준다. 서로 의존하는 스팸 패키지가 대량으로 게시되었다 [7, 9]. 레지스트리는 타이포스쿼팅과 계정 탈취를 비롯한 다른 공급망 공격도 겪으며 [30, 31, 32], 소수의 관리자 계정이 전이 의존성을 거쳐 npm의 큰 부분에 닿을 수 있다 [33].

2.4 PageRank, 재시작 분포, 시빌 공격

$G = (V, E, w)$ 를 간선 가중치가 양수인 방향 그래프라 하고, P 를 나가는 간선이 있는 노드에서는 $P_{uv} = w(u, v) / \sum_x w(u, x)$ 이고 나가는 간선이 없는 노드에서는 행 전체가 0인 행렬이라 하자. 감쇠 계수 $d(0 < d < 1)$ 와 V 위의 재시작 분포 π 가 주어지면 PageRank는 다음 식의 고정점이다.

$$\mathbf{r} = (1 - d) \pi + d P^\top \mathbf{r} + d \delta(\mathbf{r}) \pi, \quad \delta(\mathbf{r}) = \sum_{v: \text{나가는 간선 없음}} r_v \quad (2)$$

즉 나가는 간선이 없는 노드(댕글링 노드)가 가진 질량은 재시작 분포로 돌아간다 [34, 35]. 원래 알고리즘은 균등한 π 를 쓴다. 주제 민감 PageRank와 개인화 PageRank는 균등하지 않은 π 를 쓰고 [36, 37], TrustRank는 사람이 평판이 좋다고 판단한 소수의 페이지에 재시작을 몰아준다 [17]. EigenTrust는 미리 신뢰한 피어에 같은 역할을 맡긴다 [38]. AWP와 EndorseRank는 $d = 0.85$ 와 균등 재시작을 쓴다 [10, 11].

시빌 공격은 한 주체가 통제하는 신원을 많이 만드는 공격이다 [39]. PageRank를 상대로 하면 링크 팜의 꼴이 된다. 링크 팜은 대상을 가리키려고만 존재하는 페이지 집합이며, 여러 팜이 손을 잡으면 참여한 모든 팜의 점수가 오른다 [16]. Cheng과 Friedman은 노드 신원에 대해 대칭인 평판 함수는 시빌 공격에 안전할 수 없고, 신뢰하는 출발점에서 계산하는 비대칭 흐름 기반 함수는 안전할 수 있음을 보였다 [15]. SybilGuard와 SybilRank 같은 소셜 그래프 방어도 같은 관찰에 기댄다. 공격자는 노드와 그 사이의 간선을 마음대로 만들 수 있지만, 정직한 노드에서 공격자 영역으로 들어가는 공격 간선의 수는 제한되어 있고, 정직한 노드에서 출발한 길기는 이 간선을 좀처럼 건너지 않는다 [40, 18]. 커뮤니티 단위로 PageRank를 분석한 연구는 점수가 들어오는 링크에 어떻게 의존하는지를 정확히 보였고 [41], Robust PageRank는 한 노드의 점수 중 소수의 기여자에게서 올 수 있는 몫을 제한한다 [42]. 5절은 이 생각들을 컨트랙트에 적용한다.

3 관련 연구

3.1 블록체인의 그래프 기반 순위

Do와 Do는 전송 금액을 간선 가중치로 한 이더리움 거래 그래프에서 주소마다 PageRank와 HodgeRank 점수를 계산했다 [43]. AWP는 여기에 로지스틱 감쇠를 더해 최근 전송을 오래된 전송보다 무겁게 센다 [10]. RiskProp은 이분 거래 그래프를 따라 위험 라벨을 전파해 계정의 등급을 매긴다 [44]. Chen 등은 자금 흐름, 컨트랙트 생성, 컨트랙트 호출에 대한 세 가지 이더리움 그래프를 만들고 PageRank 등으로 분석했다. 이들의 호출 그래프는 이 논문이 순위를 매기는 컨트랙트 그래프에 가장 가까운 선행 대상이지만, 그 목적은 추천이 아니라 측정이었다 [45]. EndorseRank는 전송 대신 소유자와 지출자 사이의 최신 ERC-20 allowance를 썼다 [11]. 그 연구에서 지출자는 대부분 컨트랙트였으므로, allowance 그래프는 이미 주로 지갑에서 컨트랙트로 향하고 있었다. 주소 군집화 휴리스틱은 같은 소유자의

것일 가능성이 높은 이더리움 주소들을 묶으며, 그중 일부는 승인에 기반한다 [46]. 지갑 가중치를 다룰 때 이런 휴리스틱을 다시 언급한다.

이 방법들은 모두 신용이나 위험을 위해 주소의 순위를 매긴다. 개발자가 어떤 컨트랙트를 다시 써야 하는지를 묻는 방법은 없고, 공격자가 컨트랙트를 순위에서 끌어올리는 데 얼마를 내야 하는지를 다룬 연구도 없다.

3.2 의존성 네트워크와 레지스트리 보안

패키지 생태계 연구는 의존성을 방향 그래프로 보고 그 성장과 취약성을 쟀다 [27, 28]. Mujahid 등은 npm 패키지의 PageRank를 달마다 추적하고, 중심성이 떨어지는 패키지를 쇠퇴하는 패키지로 표시했다 [29]. 레지스트리 보안 연구는 타이포스쿼팅, 악의적 관리자, 계정 탈취 등 오픈소스 공급망 공격을 분류하고 [30, 31, 32], 탈취된 관리자 계정 하나가 전이 의존성을 거쳐 얼마나 멀리 닿는지 보여 준다 [33]. Sigstore 같은 서명 시스템은 게시된 산출물을 신원에 묶는다 [47]. Stacks에서는 배포자 주소가 컨트랙트 식별자의 일부이므로 컨트랙트와 게시자의 결속이 따로 비용 없이 주어진다. 그래도 레지스트리는 이름, 버전, 스코프가 그 주소와 어떻게 이어지는지를 정해야 한다.

3.3 스마트 컨트랙트의 코드 클론

스마트 컨트랙트에서는 복제가 흔하다. Kondo 등은 자신들이 살펴본 검증된 이더리움 컨트랙트의 79.2%가 클론임을 발견했고 [48], He 등은 약 천만 개의 컨트랙트 가운데 96% 이상에서 중복본을 찾았다 [49]. 클론은 사기에도 쓰인다. 인기 토큰의 복제본으로 탈중앙 거래소의 거래자를 속이는 사례가 있다 [50]. 클론 탐지 기법은 정확한 해시부터 토큰 기반·트리 기반 유사도까지 다양하며 [51], 대규모의 근사 중복 탐지에는 싱글 위의 MinHash를 흔히 쓴다 [52]. Stacks 레지스트리는 여기에 온체인을 정확한 검사를 더할 수 있다. `contract-hash?`가 어떤 컨트랙트든 그 코드의 해시를 돌려주기 때문이다 [14].

3.4 Clarity 컨트랙트를 찾는 도구

레지스트리의 몇몇 조각은 이미 있다. Stacks Blockchain API는 소스 코드와 ABI를 돌려주고 trait별로 컨트랙트를 나열한다 [3]. Clarinet은 메인넷 컨트랙트를 프로젝트 요구 사항으로 내려받고 로컬 네트워크에 맞게 주소를 바꿔 준다 [5]. 공개 GitHub 저장소 하나가 배포된 컨트랙트의 소스를 옮겨 두고 [4], Chainhooks는 체인 이벤트를 애플리케이션에 흘려보낸다 [53]. Clarity 정적 분석기 STACY는 as-contract 안에서 다른 컨트랙트를 호출하는 것 같은 패턴을 찾아낸다 [54]. 그러나 이 도구들 가운데 컨트랙트의 순위를 매기거나, 클론을 묶거나, 버전을 기록하거나, 컨트랙트를 참조하는 일과 복제하는 일을 구분하는 것은 없다.

이 연구들 가운데 배포된 컨트랙트를 재사용 관점에서 순위 매기는 연구는 없다. 가장 가까운 방법인 EndorseRank는 Stacks에 없는 allowance 관계에 의존하며, 시빌 공격에서 어떻게 행동하는지는 분석되지 않았다.

4 ERC-20 allowance에서 Stacks로

4.1 EndorseRank가 allowance에서 가져오는 것

EndorseRank는 간선 $u \rightarrow v$ 에 소유자 u 가 지출자 v 에게 내준 최신 allowance를 토큰에 걸쳐 합한 값을 싣는 그래프 G_{approve} 를 만들고, 그 위에서 가중 PageRank를 돌린다 [11]. 이것이 가능한 것은 ERC-20 allowance의 다섯 가지 성질 덕분이다.

- **자산 소유자의 명시적 행위:** 소유자가 approve 트랜잭션이나 permit 메시지에 서명한다 [55, 56].
- **지목된 상대:** 승인은 지출자를 지목하며, 지출자는 보통 라우터나 볼트 컨트랙트이다.
- **자산별 한도:** allowance는 토큰 하나의 양이다.
- **지속되는 상태:** allowance는 소유자가 바꿀 때까지 효력이 있고, 새 승인이 이전 승인을 덮어쓴다. EndorseRank는 소유자·지출자·토큰마다 최신 값만 남기므로 시간 감쇠가 필요 없다.
- **비용:** 승인에는 트랜잭션 수수료 한 번이 든다. 소유자가 승인한 양만큼 가지고 있을 필요는 없다.

마지막 성질이 뒤의 논의에서 중요하다. allowance는 만들기 싸므로, 가짜 보증에 대한 방어는 간선 자체에서 나올 수 없다.

4.2 Stacks의 후보 메커니즘

표 1에서는 ERC-20 allowance를 2.2절에서 본 Stacks의 메커니즘들과 비교한다. 다섯 성질을 모두 갖춘 메커니즘은 없다. 포스트컨디션은 명시적이고, 자산 소유자가 서명하며, 자산별 한도가 있지만, 트랜잭션 하나 동안만 유효하고 지출자를 지목하지 않는다. Clarity 4의 자산 한도는 상한을 정하지만 코드에 쓰여 있으므로 사용자가 아니라 개발자의 결정을 나타낸다. 확장 맵은 지속되고 철회할 수 있지만 대개 한도가 없다. 정적 참조는 영구적이며 자산에 대해서는 아무것도 말하지 않는다.

4.3 선택

allowance의 역할을 둘로 나눈다.

지갑→컨트랙트: 포스트컨디션. 사용자가 컨트랙트 c 를 호출하면서 자산 a 의 유출을 x 로 제한하는 트랜잭션에 서명하면, 그 사용자는 c 와, 그 트랜잭션에서 c 가 호출하는 모든 컨트랙트가 자기 계정에서 a 를 x 까지 옮기도록 허락한 것이다. 이것은 토큰 a 에 대한 $\text{approve}(c, x)$ 를 트랜잭션 하나로 좁힌 의미이다. 포스트컨디션은 지출자를 지목하지 않으므로, 승인은 트랜잭션 페이로드에 적힌 컨트랙트, 곧 사용자가 고른 진입점에 귀속한다. 이더리움에서는 사용자가 라우터를 승인하고 라우터가 토큰을 풀로 옮긴다. Stacks에서는 사용자가 라우터를 호출하고 라우터의 호출이 토큰을 옮긴다. 두 경우 모두 보증은 라우터에게 가고, 풀은 컨트랙트 간선을 따라 라우터로부터 점수를 받는다. 트랜잭션 단위의 승인에서 지속되는 상태를 얻기 위해, 지갑·컨트랙트·자산마다 EndorseRank 의 덮어쓰기 규칙을 적용한다. 가장 최근의 승인 하나만 센다(5.2절).

컨트랙트→컨트랙트: 코드 안의 참조. 컨트랙트의 소스에는 그 컨트랙트가 호출하는 컨트랙트, 구현하거나 사용하는 `trait`, 자기 권한으로 행동하게 하는 코드가 드러난다. 간선은 두 종류를 쓴다. 위임 간선 $A \rightarrow B$ 는 A 가 `as-contract?`, 옛 `as-contract`, `restrict-assets?` 안에서 B 를 호출하거나, A 의 거버넌스가 B 를 확장으로 켜다는 뜻이다. 어느 경우든 A 의 개발자나 거버넌스가 자산이나 권한을 B 에게 맡긴 것이며, 이것이 승인의

표 1: ERC-20 allowance와, 그 의미의 일부를 담는 Stacks의 메커니즘

메커니즘	주는 쪽과 받는 쪽	한도	유효 기간	만드는 비용
ERC-20 approve / permit	토큰 소유자가 지목한 지출자에게	토큰별 양	덮어쓸 때까지	수수료 한 번, 잔액 불필요
발신자 유출에 대한 포스트컨디션	트랜잭션 발신자가 호출한 컨트랙트와 그 컨트랙트가 부르는 모든 것에게	자산별 양 또는 지정한 NFT	트랜잭션 하나	수수료 한 번
as-contract? 한도 (Clarity 4)	컨트랙트 코드가 본문 안의 코드에게	호출마다 자산별 양, with-all-assets-unsafe 이면 없음	영구	배포 한 번
restrict-assets? (Clarity 4)	컨트랙트 코드가 어떤 principal을 대신해 본문 안의 코드에게	호출마다 자산별 양	영구	배포 한 번
옛 as-contract	컨트랙트 코드가 본문 안의 코드에게	없음	영구	배포 한 번
확장 맵, 허용 목록 맵	컨트랙트 거버넌스가 다른 컨트랙트에게	대개 없음	끝 때까지	거버넌스 결정
contract-call?, use-trait, impl-trait	컨트랙트 코드가 참조한 컨트랙트에게	자산과 무관	영구	배포 한 번

컨트랙트 수준 대응물이다. 의존 간선 $A \rightarrow B$ 는 A 가 정적 디스패치로 B 를 호출하거나, B 에 정의된 trait를 A 가 구현하거나 사용한다는 뜻이다. 패키지 레지스트리가 가져오기 (import)로 세는 간선이 이것이다.

빼는 것. 동적 디스패치에서는 구체적인 피호출자를 A 의 개발자가 아니라 A 를 호출하는 쪽이 고른다. 사용자가 라우터 R 에 토큰 X 를 넘기면 X 는 사용자가 고른 것이고, X 에 대한 사용자의 포스트컨디션이 이미 그것을 R 에 대한 승인으로 기록한다. 이런 실행 시점 바인딩을 간선 $R \rightarrow X$ 로 바꾸면 R 의 개발자가 한 적 없는 보증을 R 에게 돌리게 되고, 누구든 인기 있는 라우터를 호출하기만 해서 그 라우터에서 나가는 간선을 만들 수 있게

된다. 그래서 실행 시점 trait 바인딩에는 간선을 더하지 않는다. 이 간선을 더하면 무슨 일이 생기는지는 5.7절에서 본다.

4.4 승인으로 세는 포스트컨디션

포스트컨디션은 다음을 모두 만족할 때 진입 컨트랙트에 대한 승인으로 센다.

- **principal:** 트랜잭션 발신자를 제한한다. 발신자(origin) principal 유형을 쓰거나, 발신자 주소와 같은 standard principal을 쓴 경우이다. 다른 principal에 대한 포스트컨디션은 발신자를 대신해 아무것도 승인하지 않는다. 예컨대 풀 컨트랙트가 보내야 할 양의 하한은 사용자의 슬리피지 보호이다.
- **조건:** STX와 대체 가능 토큰에서는 양수를 허용하는 상한(=, <, ≤)이고, 대체 불가능 토큰에서는 *will send*나 *may send*이다. 한도가 0이면 유출을 막는 것이므로 사용자를 보호할 뿐 아무것도 승인하지 않는다. 발신자 자신의 유출에 대한 하한(>, ≥)은 양을 열어 두므로 Allow 모드의 유출처럼 다룬다.
- **결과:** 트랜잭션이 성공했다. 실패한 트랜잭션은 자산을 옮기지 않는다.

Allow 모드의 트랜잭션에서는 포스트컨디션이 다루지 않는 자산이 발신자 계정에서 제한 없이 나갈 수 있다. 이런 유출은 진입 컨트랙트에 대한 포괄 승인으로 센다. 이것은 무제한 ERC-20 승인에 해당하며, EndorseRank도 무제한 승인을 강한 보증으로 남겼다 [11]. 포괄 승인에는 표시를 해 두어 민감도 분석에서 뺄 수 있게 한다. Epoch 3.4부터 쓸 수 있는 Originator 모드도 발신자의 유출에는 포스트컨디션을 요구하므로 위 규칙에 그대로 맞는다.

4.5 AWP는 대체할 필요가 없다

AWP는 전송으로 만든다. Stacks는 STX, 대체 가능 토큰, 대체 불가능 토큰의 모든 전송을 그 전송을 일으킨 트랜잭션의 이벤트로 기록한다 [3]. 따라서 AWP는 발표된 그대로 Stacks에서 돌릴 수 있다. 전송마다 보내는 쪽에서 받는 쪽으로 간선을 두고, 금액과 로지스틱 감쇠 $\sigma(\Delta t) = 1/(1 + \exp(k(\Delta t - t_0)))$ ($k = 0.01$, $t_0 = 180$ 일)로 가중한다 [10]. 이 그래프에는 지갑과 컨트랙트가 모두 들어가지만 점수는 컨트랙트에 대해서만 보고한다. AWP는 7절에서 기준선으로 쓴다.

5 PC-EndorseRank

5.1 기호

T_0 을 순위의 동결 시점, C 를 T_0 까지 Stacks 메인넷에 배포된 컨트랙트의 집합, $\mathcal{W} = (T_0 - \Delta, T_0]$ 를 관측 기간이라 하자. ERC-20 연구와 같이 $\Delta = 180$ 일로 둔다 [11]. $\mathcal{W}$ 안의 트랜잭션 τ 에 대해 $o(\tau)$ 는 발신자(standard principal이며, 아래에서는 지갑이라 부른다), $c(\tau) \in C$ 는 페이로드가 컨트랙트 호출일 때 페이로드에 적힌 컨트랙트, $f(\tau)$ 는 수수료, $\text{out}_a(\tau)$ 는 τ 동안 $o(\tau)$ 가 보내거나 소각한 자산 a 의 양으로, 트랜잭션의 자산 이벤트에서 읽는다. $\text{Auth}(\tau)$ 는 4.4절의 규칙에 따라 τ 가 승인 포스트컨디션을 담고 있는 자산들의 집합이며, Allow 모드의 포괄 승인도 포함한다. $p(a)$ 는 공통 단위로 나타낸 자산 a 의 T_0 시점 가격이다. 믿을 만한 가격이 없는 자산과, 주 분석에서는 대체 불가능 토큰에 대해 $p(a) = 0$ 이다.

5.2 승인 기록과 최신 상태 규칙

정의 1 (승인 기록). 지갑 u , 컨트랙트 c , 자산 a 에 대해 $\tau^*(u, c, a)$ 를 $\mathcal{W}$ 안에서 $o(\tau) = u$, $c(\tau) = c$, $a \in \text{Auth}(\tau)$ 를 만족하는 가장 늦은 성공 트랜잭션이라 하자. 그 값은 $v(u, c, a) = p(a) \text{out}_a(\tau^*)$ 이다. u 의 c 에 대한 승인은 $A(u, c) = \sum_a v(u, c, a)$, u 가 승인한 컨트랙트의 집합은 $C_u = \{c : \text{어떤 } a \text{에 대해 } \tau^*(u, c, a) \text{가 존재}\}$ 이고, u 의 보증에서 c 가 차지하는 몫은 다음과 같다.

$$s_u(c) = \begin{cases} A(u, c) / \sum_{c' \in C_u} A(u, c') & \text{분모가 양수일 때,} \\ 1/|C_u| & \text{그 밖의 경우.} \end{cases} \quad (3)$$

두 가지 선택을 설명해 둔다. 첫째, allowance 값 자체를 쓰는 EndorseRank와 달리, 값은 포스트컨디션에 적힌 한도가 아니라 지갑에서 실제로 나간 양이다. 성공한 트랜잭션에서 상한은 유출보다 작을 수 없고, 한도는 ERC-20 allowance처럼 지갑이 가진 양보다 훨씬 크게 비용 없이 적을 수 있다. 실제 유출량은 자산이 움직여야 생긴다. 둘째, 최신 상태 규칙은 EndorseRank의 규칙 그대로이며, 소유자·지출자·토큰 대신 지갑·컨트랙트·자산마다 적용한다. 같은 컨트랙트를 천 번 호출한 지갑도 자기 몫 배분에는 가장 최근 호출의 값으로 한 번만 들어간다. 반복 호출은 그 호출에 든 수수료를 통해서만 지갑의 가중치를 높인다(5.4절).

5.3 재시작을 바꿔야 하는 이유

EndorseRank를 더 손대지 않고 Stacks로 옮기면, 지갑과 컨트랙트가 노드이고 간선 $u \rightarrow c$ 를 $A(u, c)$ 로 가중하며 컨트랙트 사이에도 간선이 있는 그래프에 균등 재시작을 쓰게 된다. 다음 명제는 이런 그래프가 무엇을 재는지 보여 준다.

명제 1 (균등 재시작은 지갑 수를 센다). G 의 노드 집합이 N 개 노드로 된 $W \cup C$ 이고, 지갑에서 컨트랙트로 가는 간선 $u \rightarrow c$ 의 전이 확률이 $s_u(c)$ 이며, 컨트랙트 사이에도 간선이 있고, 재시작이 균등한 $\pi_v = 1/N$ 이라 하자. 그러면 모든 지갑의 점수는 $r_W = (1 - d + d\delta)/N$ 으로 같다. 여기서 δ 는 댕글링 노드가 가진 점수이다. 또 지갑들이 컨트랙트 c 에 넘기는 점수는 $dr_W \sum_u s_u(c)$ 이다. 지갑 가운데 k 개가 저마다 컨트랙트 x 만 승인하면, 이 지갑들은 승인한 양과 관계없이 모두 합쳐 kdr_W 를 x 에 넘긴다. 이때 N, δ , 따라서 r_W 는 그 지갑들을 포함한 그래프의 값이다.

증명. 지갑으로 들어오는 간선은 없다. 컨트랙트는 트랜잭션을 발신하지 않고, 컨트랙트 사이의 간선은 컨트랙트에서 끝나기 때문이다. 따라서 식 (2)에 의해 각 지갑의 점수는 $(1 - d)/N + d\delta/N$ 이며 지갑에 따라 달라지지 않는다. 지갑 u 는 dr_W 를 나가는 간선을 따라 $s_u(c)$ 의 비율로 넘기므로, c 에 넘어가는 양의 합은 $dr_W \sum_u s_u(c)$ 이다. x 만 승인한 지갑은 $s_u(x) = 1$ 이다. $\square$

균등 재시작에서 승인한 양은 각 지갑이 정해진 기여분을 자기 컨트랙트들에 어떻게 나누는지만 정한다. 컨트랙트 점수 가운데 지갑에서 오는 부분은, 지갑마다 한 표를 자기가 쓰는 컨트랙트들에 나누어 준 뒤 센 지갑 수이다. 시빌 지갑 하나는 트랜잭션 수수료 한 번으로 온전한 한 표를 가져온다. 금액으로 가중해도 이것은 바뀌지 않는다. 이 명제는 ERC-20 연구에서 단순 입차수가 EndorseRank보다 떼어 둔(held-out) 승인과 더 잘 맞은 까닭도 하나 시사한다 [11]. 지출자이면서 소유자이기도 한 주소가 적은 allowance 그래프는 이분 그래프에 가깝고, 그런 그래프에서 걷기는 개수에 보태는 것이 거의 없다.

5.4 지갑 가중치와 재시작 분포

그래서 지갑을 노드 집합에서 빼고, 공격자가 돈을 내야 얻을 수 있는 가중치 $\omega(u)$ 를 붙여 재시작 분포로만 작용하게 한다.

$$\omega(u) = \sum_{\tau \in \mathcal{W}: o(\tau)=u} \min(f(\tau), f_{\max}), \quad \pi_c = \frac{\sum_u \omega(u) s_u(c)}{\sum_{u: C_u \neq \emptyset} \omega(u)} \quad (4)$$

지갑의 가중치는 그 지갑이 관측 기간 안에서 발신한 모든 트랜잭션의 수수료를 합한 값이며, 각 수수료는 $\mathcal{W}$ 안 수수료의 99번째 백분위수인 $f_{\max}$ 에서 자른다. 대납자가 낸 수수료도 발신자의 것으로 센다. 누군가는 그 수수료를 냈고, 자기 지갑의 수수료를 대납하는 공격자는 그 돈을 스스로 낸다. 식 (4)는 ω 에 비례해 고른 지갑에서 출발해 승인 간선을 한 걸음 따라간 걸기가 보게 되는 재시작 분포와 같다.

명제 2 (수수료 예산을 나눠도 얻는 것이 없다). 공격자가 x 만 승인하는 지갑들로 수수료 예산 F 를 쓴다고 하자. 식 (4)에서 이 지갑들이 π_x 의 분자에 보태는 양은 예산을 지갑들에 어떻게 나누든 F 이하이다. 반면 $g(0) = 0$ 인 오목 함수 g 로 $\omega(u) = g(u \text{의 수수료})$ 를 쓰면, F 를 k 개 지갑에 고르게 나눌 때 $k g(F/k) \geq g(F)$ 를 보탠다. $\omega \equiv 1$ 이면 최소 수수료 k 번의 비용으로 k 를 보탠다.

증명. 첫 주장은 선형성과, 센 수수료를 낮출 수만 있는 상한에서 나온다. $g(0) = 0$ 인 오목 함수 g 는 준가법적이므로 $g(F) = g(\sum_{i=1}^k F/k) \leq \sum_{i=1}^k g(F/k)$ 이다. $\square$

선형 가중치에서는 영향력이 트랜잭션에 쓴 돈에 비례하고, 공격자는 신원을 늘려서 이 비용을 피할 수 없다. 그 대가로 거래를 많이 하는 지갑을 드물게 하는 지갑보다 더 무겁게 센다. 지갑 하나에 한 표를 주는 규칙에는 Stacks에 없는 신원 계층이 필요하며, Cheng과 Friedman의 결과에 따르면 대칭적인 규칙으로는 이 문제를 피할 수 없다 [15]. $\omega \equiv 1$ 에 맞서는 공격자는 최소 수수료 한 번으로 영향력 한 단위를 산다. 식 (4)에서는 정직한 지갑 n 개의 영향력을 따라잡으려면 그 지갑들이 낸 만큼을 내야 하므로, 비용은 정직한 지갑의 평균 수수료와 최소 수수료의 비만큼 오른다. 7절에서 이 비를 메인넷에서 잰다. 민감도 분석에서는 $\mathcal{W}$ 동안 지갑의 STX 잔액을 시간 평균한 자본-일도 쓴다. 자본-일은 수수료처럼 나눠도 합이 변하지 않고, 같은 코인이 동시에 두 지갑에 있을 수도 없다.

Stacks 채굴자는 트랜잭션 수수료를 받으므로, 채굴자는 수수료를 자기 자신에게 낼 수 있다. 상한 $f_{\max}$ 는 스스로 낸 트랜잭션 하나가 살 수 있는 양을 제한한다. 이 문제는 8절에서 다시

다룬다.

5.5 컨트랙트 사이의 간선

E_{del} 과 E_{dep} 을 4절의 위임 간선과 의존 간선이라 하자. 이 간선들은 소스와 거버넌스 트랜잭션에서 다음과 같이 뽑는다.

- 의존: A 에 대상 B 를 절대 principal, 상대 principal .name(같은 배포자의 컨트랙트), 그런 principal에 묶인 상수(Clarify 5) 중 하나로 쓴 contract-call?이 있거나, A 의 impl-trait 또는 use-trait가 B 에 정의된 trait를 가리키면 $A \rightarrow B$ 이다.
- 위임: 대상을 알아낸 A 에서 B 로의 호출이 as-contract?, 옛 as-contract, restrict-assets?의 본문 안에 있거나, A 가 ExecutorDAO 패턴을 따르고 A 의 이력에서 가장 최근의 set-extension 호출이 B 를 켜 두었으면 $A \rightarrow B$ 이다.
- 제외: 대상이 trait 타입 인자인 호출, 자기 자신으로 가는 간선, T_0 에 존재하지 않는 컨트랙트에 대한 참조는 뺀다.

두 집합에 모두 든 간선은 위임으로 센다. 가중치는 $\beta_{\text{del}} = 2$, $\beta_{\text{dep}} = 1$ 로 미리 정하고, 민감도 분석에서 β_{del} 을 $\{1, 2, 4\}$ 로 바꾼다. 위임을 더 무겁게 두는 것은 위임이 자산이나 권한을 피호출자에게 맡기기 때문이며, 승인과 사용을 구분하는 것이 EndorseRank의 바탕이다. 전이 행렬은 $P_{AB} = \beta(A, B) / \sum_{B'} \beta(A, B')$ 이다.

5.6 점수

PC-EndorseRank는 노드 집합 C , 전이 행렬 P , 식 (4)의 재시작 π 로 식 (2)를 푼 해이다. ERC-20 연구와 같이 $d = 0.85$, ℓ_1 허용 오차 10^{-8} , 최대 300회 반복을 쓴다 [11]. 알고리즘 1에 단계를 적었다. 점수가 0인 컨트랙트는 사용된 근거가 없는 컨트랙트이며, 레지스 트리는 이들을 목록에는 올리되 순위는 매기지 않는다.

명제 3 (공격자 컨트랙트의 점수). 집합 $S \subseteq C$ 에 대해 $\Phi_{\text{in}}(S) = \sum_{v \notin S} r_v P_{vS}$ 를 S 밖의 컨트랙트에서 간선을 따라 S 로 흘러드는 점수라 하자. 여기서 $P_{vS} = \sum_{y \in S} P_{vy}$ 이다. 그러면 다음이 성립한다.

$$r(S) \leq \frac{\pi(S)}{1-d} + \frac{d}{1-d} \Phi_{\text{in}}(S) \quad (5)$$

특히 $\pi(S) = 0$ 이고 S 밖의 어떤 컨트랙트도 S 로 가는 간선을 갖지 않으면 $r(S) = 0$ 이다.

증명. 식 (2)를 S 에 대해 더하면 $r(S) = (1-d)\pi(S) + d\sum_v r_v P_{vS} + d\delta\pi(S)$ 이다. 나가는 간선이 있는 S 의 노드는 많아야 자기 점수 전부를 S 로 보내고, 댕글링 노드는 아무것도 보내지 않으므로, S 안의 댕글링 점수를 δ_S 라 하면 $\sum_{v \in S} r_v P_{vS} \leq r(S) - \delta_S$ 이다. 따라서 $(1-d)r(S) \leq (1-d)\pi(S) + d\Phi_{\text{in}}(S) + d(\delta\pi(S) - \delta_S)$ 이다. $\delta \leq 1$ 이고 $\delta_S \geq 0$ 이므로 마지막 항은 $d\pi(S)$ 이하이며, 여기서 상한이 나온다. $\pi(S) = 0$ 이고 $\Phi_{\text{in}}(S) = 0$ 이면 부등식은 $r(S) \leq 0$ 이 된다. $\square$

우변의 두 항은 공격자가 얻어야 하는 두 가지이다. $\pi(S)$ 는 S 안의 컨트랙트를 승인하는 지갑들이 낸 수수료로 사며, 그 값은 명제 2가 정한다. $\Phi_{\text{in}}(S)$ 를 얻으려면 이미 점수가 있는 컨트랙트가 코드에서 S 를 참조하거나 S 를 확장으로 켜야 한다. 이것은 다른 개발자와 거버넌스가 내리는 결정이며, 소셜 그래프 방어에서 말하는 공격 간선에 해당한다 [40, 18]. 대상을 참조하는 컨트랙트를 배포하면 S 의 구성원은 늘지만 어느 정도 커지지 않는다. 반면 균등 재시작에서는 배포한 컨트랙트마다 $1/|C|$ 의 재시작 질량을 함께 가져오므로, 공격자가 배포한 컨트랙트 수만큼 상한이 커진다.

알고리즘 1 PC-EndorseRank

입력: T_0 시점의 소스가 있는 컨트랙트 C ; $\mathcal{W}$ 안의 트랜잭션; 가격 p ; 매개변수 $d, \beta_{\text{del}}, \beta_{\text{dep}}, f_{\text{max}}$

출력: C 위의 점수 $\mathbf{r}$

- 1: 각각에 대해: $\mathcal{W}$ 안의 성공한 컨트랙트 호출 τ 와 $a \in \text{Auth}(\tau)$ 의 모든 쌍
 - 2: τ 가 저장된 것보다 늦으면 $\tau^*(o(\tau), c(\tau), a)$ 로 저장
 - 3: 끝
 - 4: $A(u, c) \leftarrow \sum_a p(a) \text{out}_a(\tau^*(u, c, a))$; 식 (3)에 따라 $s_u(c)$ 계산
 - 5: $\omega(u) \leftarrow \sum_{\tau: o(\tau)=u} \min(f(\tau), f_{\text{max}})$; 식 (4)에 따라 π 계산
 - 6: 각 컨트랙트를 파싱; 리터럴·상대·상수 대상을 풀어 E_{dep} 과 E_{del} 을 모음; 켜진 확장을 E_{del} 에 더함
 - 7: $P_{AB} \leftarrow \beta(A, B) / \sum_{B'} \beta(A, B')$
 - 8: $\mathbf{r} \leftarrow \pi$
 - 9: 반복
 - 10: $\mathbf{r}' \leftarrow (1-d)\pi + dP^\top \mathbf{r} + d\delta(\mathbf{r})\pi$; $\mathbf{r}$ 과 $\mathbf{r}'$ 의 값을 맞바꿈
 - 11: 멈춤 조건: $\|\mathbf{r} - \mathbf{r}'\|_1 < 10^{-8}$ 또는 300회 반복
 - 12: 반환 $\mathbf{r}$
-

5.7 합성 예제

명제들이 실제로 어떻게 작동하는지 보이려고 작은 합성 그래프를 만들었다(스크립트는 논문 저장소에 있고, 매개변수는 부록 B에 적었다). 정직한 컨트랙트 14개에는 SIP-010 trait 하나, 토큰 여섯 개, as-contract? 안에서 토큰을 내주는 풀 세 개, 라우터, 볼트, DAO가 들어 있다. 정직한 지갑 300개는 저마다 진입 컨트랙트를 한 개에서 세 개 승인하고, 관측 기간 동안 평균 최소 수수료의 31.1배를 낸다. 공격자는 토큰 x 를 배포하고 7절의 공격 가운데 세 가지로 x 를 상위 3위 안에 올리려 한다. A1은 x 를 호출하는 컨트랙트를 배포한다(컨트랙트 하나에 수수료 10단위). A2는 최소 수수료로 x 를 한 번 승인하는 지갑을 만든다. A4는 x 를 trait 인자로 넣어 정직한 라우터를 호출한다. 왕복 송금인 A3은 AWP에만 통하므로 이 예제에서는 뺐다. 표 2에 다섯 가지 점수 규칙에서 상위 3위에 드는 데 필요한 최소 비용을 적었다.

표 2: 정직한 컨트랙트 14개로 된 합성 그래프에서 공격자 토큰을 상위 3위 안에 올리는 최소 공격 비용(최소 호출 수수료 단위). 공격 크기는 두 배씩 늘려 시험한 뒤 이분 탐색으로 좁히며, 공격이 커져도 토큰의 순위가 내려가지 않는다고 가정한다. 동점은 토큰에 유리하게 센다. “도달 못 함”은 공격 크기가 200,000일 때도 토큰이 상위 3위 밖에 있다는 뜻이다. 그래프는 꾸며 낸 것이며, 이 표는 명제 1-3의 예시일 뿐 Stacks 메인넷에 대한 결과가 아니다.

점수 규칙	A1	A2	A4
서로 다른 호출자 수	도달 못 함	59	도달 못 함
의존성 PageRank, 균등 재시작	10	도달 못 함	도달 못 함
Stacks로 옮긴 EndorseRank, 균등 재시작	750	75	도달 못 함
PC-EndorseRank(수수료 가중 재시작)	도달 못 함	2,234	도달 못 함
PC-EndorseRank + 실행 시점 trait 바인딩	도달 못 함	2,234	3,952

기준선은 저마다 적어도 한 가지 공격에 무너진다. 호출자 수 세기는 지갑 팜(A2)에 59단위로 무너진다. teaRank와 같은 풀인 균등 재시작 의존성 PageRank는 팜 컨트랙트 하나(A1, 10단위)에 무너진다. 배포한 컨트랙트마다 자기 몫의 재시작 질량을 가져오기 때문이다. 균등 재시작 EndorseRank는 두 공격 모두에 무너진다. 명제 1의 예측대로 지갑 75개면 충분해서 호출자 수 세기에 필요한 59개와 큰 차이가 없고, 팜 컨트랙트도 재시작 질량을 가진 노드이므로 통한다. PC-EndorseRank는 명제 3에서 보듯 A1에는 크기와 관계없이 움직이지 않고, A2에는 2,234단위가 들어 균등 재시작일 때의 약 30배이다. 이 배수는 5.4절의 예측대로 정직한 지갑이 낸 평균 수수료(최소 수수료의 31.1배)에 가깝다. A4는 실행 시점 trait 바인딩을 라우터 간선으로 더했을 때만 토큰을 움직인다(3,952단위). 이 그래프에서는 이 공격이 A2보다 비싸다. 정직한 호출이 이미 라우터의 토큰을 수백 번 바인딩하고 있기 때문이다. 점수가 호출보다 위임에서 주로 오는 라우터라면 더 싸게 장악할 수 있다.

5.8 클론 묶음, 패키지, 검색

클론 묶음. 정규화한 소스의 해시가 같은 두 컨트랙트는 같은 묶음에 속한다. 정규화는 주석과 공백을 지우고, 문자열·숫자 리터럴, `principal` 리터럴, 정의한 토큰·맵·변수의 이름을 자리표시자로 바꾼 뒤 토큰 열을 해시한다. 코드 본문이 똑같은 컨트랙트들은 `contract-hash?` 값도 같다. MinHash [52]로 찾은 Jaccard 유사도 0.9 이상의 근사 중복은 관련 컨트랙트로 보여 주되 합치지는 않는다. 묶음은 점수가 가장 높은 구성원이 대표하고(동점이면 먼저 배포된 것), 묶음의 순위는 그 구성원의 점수로 정한다. 합 대신 최댓값을 쓰므로, 어떤 컨트랙트의 복제본이 천 개 있어도 그 묶음에는 아무것도 보태지 않는다.

패키지. 패키지는 게시자가 선언한 컨트랙트의 계보이다(6절). 패키지의 순위는 최신 버전의 점수로 정하고, 이전 버전은 그 아래에 나열한다.

검색. 식 (1)의 꼴을 따라, 질의 q 와 맞는 컨트랙트 c 에 다음 점수를 준다.

$$\text{score}(q, c) = \text{BM25}(q, c) \cdot (\ln(e + |C| r_c))^\alpha, \quad \alpha = 1 \quad (6)$$

여기서 BM25 [57]는 컨트랙트 이름, 게시자 스코프, ABI의 함수 이름과 `trait` 이름, 소스의 주석, 패키지 README를 대상으로 계산하고, $|C| r_c$ 는 점수가 평균인 컨트랙트에서 1이다. 결과는 `trait` 준수 여부(SIP-009, SIP-010, SIP-013), Clarity 버전, 네트워크로 거를 수 있다.

6 레지스트리와 cpm

6.1 구성

그림 1에 구성 요소를 보였다. 인덱서는 컨트랙트, 트랜잭션, 포스트컨디션, 자산 이벤트를 PostgreSQL 데이터베이스에 채운다. 과거 기록은 호스팅 서비스 대신 직접 운영하는 Stacks 노드와 직접 운영하는 Stacks Blockchain API에서 읽는다. 호스팅 서비스로 전체 이력을 채우면 요청 한도 때문에 느리고, 고정된 보관본이 있어야 평가를 재현할 수 있기 때문이다. 새 블록은 Chainhooks나 노드의 이벤트 인터페이스로 받는다 [3, 53]. 분석기는 컨트랙트를 하나씩 파싱해 5절의 참조와 자산 한도를 뽑고, 정규화 해시를 계산하고, 정적 검사를 돌린다. 매일 도는 작업이 그래프를 만들고 PC-EndorseRank와 클론 묶음을 계산하고 검색

색인을 새로 고친다. 웹 화면과 cpm 클라이언트는 같은 검색 API를 읽으며, cpm은 받은 내용을 체인과 대조한다.

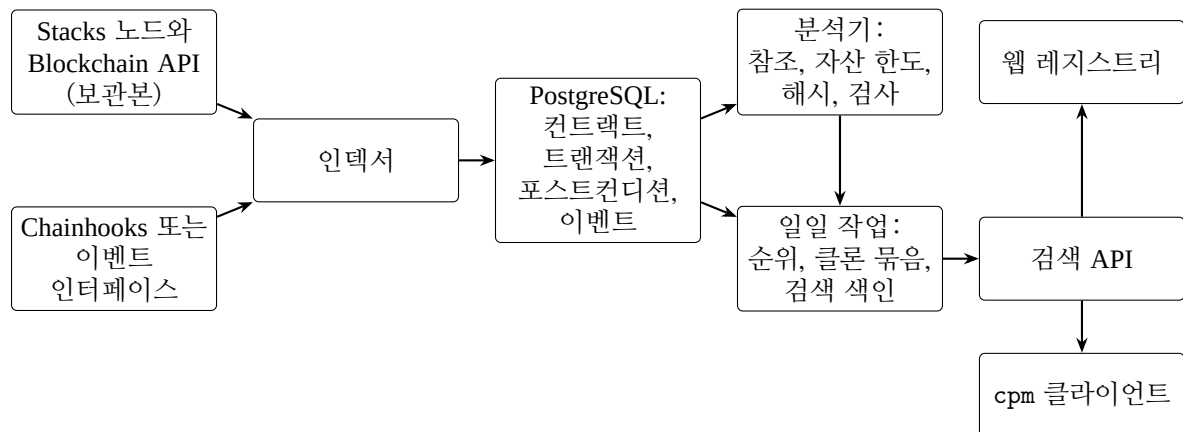


그림 1: Clarity Registry의 구성 요소. cpm 클라이언트는 API가 돌려준 내용을 확인하려고 Stacks 노드에서 코드 해시를 직접 읽기도 한다.

6.2 패키지 모형

스코프. 모든 컨트랙트에는 이미 게시자가 있다. `principal` 안의 주소가 그것이다. 그래서 레지스트리는 모든 컨트랙트를 그 `principal` 아래에 색인하고, 게시자는 읽기 쉬운 스코프를 두 가지 방법으로 등록할 수 있다. 주소 자체는 언제나 유효한 스코프이다. `alice.btc` 같은 BNS 이름은 등록을 신청할 때 그 이름이 배포자 주소로 해석되면 스코프가 된다. BNS 이름은 SIP-009 토큰이어서 팔 수 있으므로 [58], 레지스트리는 이름, 그 소유자, 신청한 블록 높이를 기록한다. 나중에 이름의 주인이 바뀌면 스코프는 동결된다. 기존 버전은 신원을 그대로 유지하고, 새 버전을 올리려면 새로 신청해야 하며, 스코프의 소유자가 프로젝트 잠금 파일에 기록된 소유자와 다르면 cpm이 경고한다. 이것은 다른 레지스트리에서 알려진 공격인, 신뢰받는 이름의 탈취를 막는다 [31].

버전. 배포된 컨트랙트는 바뀌지 않으므로 패키지의 버전마다 별개의 컨트랙트가 되고, 체인에서 `token-v1`과 `token-v2`는 서로 관계가 없다. 게시자는 그 관계를 매니페스트에 적는다. 패키지 이름, 각 컨트랙트의 시맨틱 버전, 각 버전의 이전 버전, 대응하는 테스트넷 `principal`, 라이선스 식별자, 저장소 URL, README, 감사 보고서 링크가 들어간다. 레지스트리는 나열된 모든 컨트랙트가 스코프의 주소로 배포되었고, 매니페스트가 그 주소의 SIP-018 구조화 데이터 서명을 갖추었거나 [59] 그 주소에서 보낸 트랜잭션으로 제출된 경우에만 매니페스트를 받아들인다. 서명하는 주소가 곧 배포한 주소이므로 다른 키 기반 구조는 필요 없다.

등록되지 않은 컨트랙트. 매니페스트가 없는 컨트랙트도 principal 아래에 trait 표시, 클론 묶음, 점수와 함께 색인되지만, 버전과 README는 없다.

6.3 명령

cpm은 Clarinet을 감싸는 도구이며 Clarinet을 대신하지 않는다. 표 3에 명령을 나열했다. 핵심은 참조와 복제의 구분이다. npm에서는 패키지를 설치하면 코드가 프로젝트로 복사된다. 프로젝트가 사용하려는 Clarity 컨트랙트는 이미 배포되어 모두가 함께 쓰며, principal로 호출된다. 프로젝트가 더하는 것은 참조이다. cpm add는 그 참조를 Clarinet 요구 사항으로 기록해서, 로컬 테스트는 Clarinet이 로컬 네트워크에 배포한 사본을 호출하고 메인넷 배포 때는 그 컨트랙트에 대해 아무것도 배포하지 않게 한다 [5]. SIP-010 템플릿으로 만든 토큰처럼 자기 인스턴스가 필요한 프로젝트는 cpm create나 cpm fork로 소스를 복제해 자기 주소로 배포한다. 복제는 패키지의 라이선스가 허락할 때만 할 수 있다. 체인에 공개되었다고 오픈소스가 되는 것은 아니다.

표 3: cpm 클라이언트의 명령

명령	하는 일
cpm search <query>	식 (6)에 따라 검색한다. trait, Clarity 버전, 네트워크로 거를 수 있다.
cpm info <pkg>	버전, principal, 클론 묶음, 점수, 표 4의 신호를 보여 준다.
cpm add <pkg>[@ver]	배포된 컨트랙트를 참조한다. 그 컨트랙트와 정적 의존성을 찾고, 프로젝트와 Clarity 버전·에포크를 맞춰 보고, Clarinet 요구 사항을 더하고, 잠금 파일을 쓴다. cpm install은 별칭이다.
cpm create <template> <name>	템플릿의 소스를 contracts/로 복사해 이름을 바꾸고, 선언된 매개변수를 채우고, 템플릿의 묶음 해시를 기록한다. 라이선스가 복제를 허락하지 않으면 거부한다.
cpm fork <pkg>	라이선스가 허락하는 패키지에 대해 create와 같은 일을 한다.
cpm why <contract>	프로젝트의 컨트랙트에서 주어진 컨트랙트까지의 의존 경로를 출력한다.
cpm audit	모든 의존성에 대해 표 4의 신호를 보고한다.
cpm verify	잠긴 컨트랙트마다 노드에서 코드 해시를 읽어 잠금 파일과 비교한다.

6.4 잠금 파일

npm 잠금 파일은 package.json의 버전 범위가 실제로 풀린 정확한 버전을 고정한다. 배포된 컨트랙트에는 풀어야 할 버전이 없으므로, cpm 잠금 파일은 신원을 기록한다. 의존성마다

메인넷 principal과 대응하는 테스트넷 principal, contract-hash?가 돌려주는 코드 해시, 배포 높이와 Clarity 버전, 묶음 해시, 라이선스, 추가할 때 본 스코프 소유자, 참조했는지 복제했는지를 적는다. cpm verify는 아래의 레지스트리 컨트랙트처럼 contract-hash?를 돌려주는 컨트랙트에 읽기 전용 호출을 보내 노드에서 코드 해시를 읽고, 잠금 파일과 비교한다. 그래서 API나 미러가 탈취되더라도 프로젝트가 의존하는 컨트랙트의 소스를 다른 것으로 바꿔치기할 수 없다.

6.5 선택적인 레지스트리 컨트랙트

이름을 체인에 둘 수도 있다. 작은 레지스트리 컨트랙트가 publish(name, version, contract)를 제공한다. 이 함수는 principal-destruct?로 contract의 주소 부분이 tx-sender와 같은지 확인하고, 그 컨트랙트의 contract-hash? 결과를 이름·버전과 함께 저장하고, 인덱서를 위해 이벤트를 출력한다. 아무 principal에 대해서나 contract-hash?를 돌려주는 읽기 전용 함수도 제공하며, cpm은 이를 검증에 쓴다. 레지스트리는 이 컨트랙트 없이도 작동한다. 이 컨트랙트가 있으면 이름과 해시가 체인에 기록되어 웹 서비스 운영자에게 의존하지 않는다.

6.6 사용자에게 보여 주는 신호

순위는 결과의 순서를 정할 뿐, 컨트랙트가 안전하다고 말하지 않는다. 컨트랙트 페이지와 cpm audit은 점수 옆에 표 4의 신호를 보여 주며, 레지스트리는 색인과 추천을 분리한다. 모든 컨트랙트를 색인한다. “추천” 표시를 둔다면 감사 링크, 최소 경과 기간, 심각한 지적 사항 없음처럼 점수 밖의 기준을 요구하며, 레지스트리는 점수만으로 컨트랙트를 악성이라 부르지 않는다.

7 평가 설계

평가 계획은 ERC-20 연구에서 사전 등록한 채현 연구와 같이 라벨 기간이 열리기 전에 고정한다 [11]. 아래의 날짜, 매개변수, 난수 시드, 결정 규칙을 담은 설정 파일은 2026년 11월 1일 전에 커밋하고, 그 해시를 결과와 함께 보고한다.

표 4: 컨트랙트마다 함께 보여 주는 신호

신호	중요한 이유
with-all-assets-unsafe, 또는 다른 컨트랙트 호출을 감싼 옛 as-contract	컨트랙트가 다른 코드에 한도 없이 자기 자산을 옮기게 한다. STACY는 as-contract 안의 호출을 찾아낸다 [54].
as-contract? 안에서 호출하는 trait 타입 인자	컨트랙트의 자산이 호출자가 고른 코드에 노출된다.
Allow 모드 호출의 비율	이 컨트랙트의 사용자들이 포스트컨디션 보호 없이 트랜잭션에 서명하고 있다.
클론 묶음과 대표	이 컨트랙트가 묶음에서 가장 많이 쓰이는 구성원인지, 많은 복제본 가운데 하나인지 보여 준다.
경과 기간, Clarity 버전, 에포크	오래된 Clarity 버전에는 자산 한도가 없다. 아주 새로운 컨트랙트는 이력이 짧다.
정적 분석 지적 사항 감사 링크	STACY와 레지스트리 자체 검사의 결과. 매니페스트에서 가져온다. 레지스트리는 감사 자체를 검증하지 않는다.

7.1 연구 질문

- **RQ1(신호)**: Stacks 트랜잭션은 얼마나 자주 승인 포스트컨디션을 담고 있으며, 컨트랙트는 얼마나 자주 자산 한도, 옛 as-contract, 확장 맵을 쓰는가?
- **RQ2(공격 비용)**: 공격자가 각 순위에서 컨트랙트를 상위로 끌어올리려면 얼마를 내야 하는가?
- **RQ3(채택)**: 동결 시점의 PC-EndorseRank는 이후의 채택과 가장 좋은 기준선만큼 잘 맞는가? 컨트랙트 사이의 간선은 재시작 분포가 주는 것 이상의 정보를 더하는가?
- **RQ4(잡음)**: 배포된 컨트랙트 가운데 복제본이거나 쓰인 흔적이 없는 것의 비율은 얼마이며, 묶음 단위로 모으면 검색 결과가 얼마나 달라지는가?

7.2 데이터와 기간

모든 데이터는 직접 운영하는 Stacks 노드와 직접 운영하는 Stacks Blockchain API를 통해 Stacks 메인넷에서 얻는다. 순위는 $T_0 = 2026$ 년 10월 31일 23:59:59 UTC에 동결한다. 컨트랙트 그래프에는 T_0 까지 배포된 모든 컨트랙트를 쓴다. 승인과 수수료는 T_0 전 180일 동안의 것을 쓴다. 이 관측 기간은 Epoch 3.4(비트코인 블록 943,333, 2026년 4월) 뒤에 시작하므로 기간 전체에서 Originator 모드를 쓸 수 있다 [20, 24]. Epoch 4.0(비트코인 블록 960,230, 2026년 7월)은 관측 기간 안에 있다. 이 에포크는 Clarity 자산 한도 하나의 이름을

바꾸고 하나를 더했지만 새로운 종류의 트랜잭션 포스트컨디션은 더하지 않았으므로, 4.4절의 규칙은 관측 기간 전체에 그대로 적용된다 [25]. 라벨은 2026년 11월 1일부터 2027년 1월 31일까지의 기간 L 에서 얻는다. 재현은 $T'_0 = 2027$ 년 1월 31일로 모든 것을 반복하고, 라벨은 2027년 2월 1일부터 4월 30일까지의 기간에서 얻는다. 가격은 설정 파일에 적은 공개 시장 출처에서 T_0 전 30일 동안의 일별 가격 중앙값을 쓴다. 그래야 T_0 에 맞춘 짧은 가격 펌핑이 거의 영향을 주지 못한다.

7.3 비교하는 방법

표 5에 방법을 나열했다. 단순 개수를 넣는 까닭은 1절에서 밝혔다. 새 순위는 단순 개수를 이기거나, 적어도 그보다 나쁘지 않으면서 공격자에게 더 큰 비용을 지운다는 것을 보여야 한다.

표 5: 비교하는 방법. 모든 PageRank 변형은 $d = 0.85$, ℓ_1 허용 오차 10^{-8} , 최대 300회 반복을 쓴다.

방법	정의
PC-EndorseRank	5절. 컨트랙트가 노드이고, 위임 간선과 의존 간선을 쓰며, 수수료로 가중한 승인에서 재시작 분포를 만든다.
B1 호출자 수	관측 기간 안에서 그 컨트랙트를 승인하며 호출한 서로 다른 지갑의 수.
B2 의존 컨트랙트 수	그 컨트랙트로 간선이 있는, 다른 배포자의 서로 다른 컨트랙트 수. pkg.go.dev의 가져오는 패키지 수에 해당한다 [6].
B3 의존성 PageRank	컨트랙트가 노드이고 모든 참조의 가중치가 1이며 재시작이 균등하다 (teaRank와 같은 꼴).
B4 AWP	로지스틱 감쇠($k = 0.01$, $t_0 = 180$ 일)를 쓰는 지갑·컨트랙트 전송 그래프와 균등 재시작 [10]. 컨트랙트 점수만 보고한다.
B5 Stacks로 옮긴 EndorseRank	지갑과 컨트랙트가 노드이고, $A(u, c)$ 로 가중한 간선 $u \rightarrow c$ 와 컨트랙트 간선을 쓰며, 재시작이 균등하다.
결제 분석	점수를 π 로만 매김(컨트랙트 간선 없음); 실행 시점 trait 바인딩을 간선으로 더함; Allow 모드 승인을 뺌; $\omega \equiv 1$; 자본-일로 ω 를 정함; 가격 대신 원래 토큰 단위를 씀; $\beta_{\text{del}} \in \{1, 4\}$.

7.4 채택 라벨

어떤 컨트랙트가 중요한지 알려 주는 정답은 없다. ERC-20 연구에서처럼 관측할 수 있는 결과를 라벨로 쓰고, 라벨마다 순위 일치를 보고한다.

- **Y1 신규 호출자:** 그 컨트랙트를 처음으로 승인하며 호출한 시점이 L 안에 있는 지갑

의 수. 걸러 낸 지표에서는 L 안에서 서로 다른 날짜 사흘 이상에 트랜잭션을 발신한 지갑만 센다.

- **Y2 신규 의존 컨트랙트:** 다른 배포자가 L 안에 배포했으며 그 컨트랙트로 위임 간선이나 의존 간선이 있는 컨트랙트의 수.
- **Y3 개발자 채택:** `Clarinet.toml`에 그 컨트랙트를 요구 사항으로 처음 적은 시점이 L 안에 있는 공개 GitHub 저장소의 수. T_0 과 L 의 끝에 찍은 코드 검색 스냅샷으로 구한다. 0이 아닌 값을 가진 컨트랙트가 100개 이상일 때만 가설 검정에 쓴다.
- **Y4 선별 집합:** T_0 시점에 DefiLlama가 Stacks용으로 나열한 프로토콜의 컨트랙트 [60], 또는 T_0 전 날짜의 공개 감사 보고서가 다룬 컨트랙트. 점수를 계산하기 전에 설정 파일에 적어 둔 규칙으로 모은다.

Y1은 구성상 B1과 가깝다. ERC-20 연구의 전송 점점이 AWP와 가까웠던 것과 같다. 따라서 두 값의 일치는 독립적인 예측력이 아니라 일관성을 보여 준다. 순위 일치는 근거 집합, 곧 관측 기간 안에 승인이 한 번 이상 있거나 T_0 에 들어오는 간선이 하나 이상 있는 컨트랙트에서 Kendall의 τ_b [61]와 Spearman의 ρ [62]로 계산한다. Y1과 Y2를 등급 이득으로 쓴 NDCG@100 [63]과, Y4에 대한 상위 100개 정밀도와 ROC 곡선 아래 면적도 보고한다.

7.5 공격 모의실험

동결한 데이터의 사본에 공격을 넣고, 바뀐 사본에서 모든 방법을 다시 계산한다.

- **A1 컨트랙트 팜:** 대상을 정적 디스패치로 호출하는 새 컨트랙트 k 개.
- **A2 지갑 팜:** 새 지갑 k 개가 저마다 아래에 정의한 호출 수수료 c_{call} 로 대상에 승인 호출을 한 번 보낸다. AWP에 대해서는 호출마다 가격이 있는 가장 작은 양을 대상에게 전송하기도 한다.
- **A3 왕복 송금:** 지갑 하나가 대상에게 자산을 보냈다가 돌려받기를 m 번 한다. 대상은 공격자가 통제한다.
- **A4 바인딩 주입:** SIP-010 trait 인자를 받는 라우터 가운데 점수가 가장 높은 것을, 대상을 그 인자로 넣어 m 번 호출한다. 실행 시점 바인딩을 간선으로 더한 절제 분석에만 관련된다.

비용은 STX로 낸다. 배포 한 번의 비용은 관측 기간 안에서 2KB 미만인 컨트랙트 배포의 수수료 중앙값이다. 모의 호출은 모두 관측 기간 안의 컨트랙트 호출 수수료 가운데 5번째 백분위수인 c_{call} 로 보내며, 그 비용도 c_{call} 이다. 공격자에게 돌아오는 자본은 비용으로 세지 않는다. 모든 방법의 순위에서 아래쪽 절반에 있는 근거 집합 컨트랙트 가운데 무작위로 고른 대상 200개마다, 대상을 전체 상위 100위(근거 집합이 2,000개 미만이면 상위 5%)와 그 trait 범주의 상위 10위 안에 올리는 가장 작은 k 나 m 을 두 배씩 늘리기와 이분 탐색으로 찾는다. 공격이 100,000 STX 예산 안에서 움직이지 못하는 방법에는 그 예산을 비용으로 준다. 이렇게 하면 그 방법의 이점은 실제보다 작게 잡힌다.

7.6 가설과 결정 규칙

- **H1(공격 비용):** 공격이 기준선을 움직이는 모든 짝, 곧 A1에 대한 B2, B3, B5와 A2에 대한 B1, B4, B5에서, PC-EndorseRank의 비용과 기준선의 비용 사이의 로그 비의 대상별 중앙값이 양수이고 그 95% 부트스트랩 신뢰구간의 하한이 0보다 크다. A3에 대한 B4는 검정 없이 보고한다.
- **H2(채택, 비열등성):** PC-EndorseRank의 Y1과 Y2(조건을 갖추면 Y3도)에 대한 τ_b 평균이, 가장 좋은 기준선의 평균에서 0.02를 뺀 값 이상이다. 판정은 차이의 짝지은 95% 부트스트랩 구간의 하한으로 한다. 이 허용 폭은 ERC-20 재현에서 쓴 것과 같다 [11].
- **H3(전파):** Y2에 대한 τ_b 가 PC-EndorseRank에서 π 만으로 점수를 매기는 절제 분석보다 높고, 짝지은 구간의 하한이 0보다 크다.

이 논문이 PC-EndorseRank가 채택과의 일치를 잃지 않으면서 조작 비용을 높인다고 주장하는 것은 주 기간에서 H1과 H2가 모두 성립할 때뿐이다. H3은 부차적이다. 모든 계수는 방향과 관계없이 재현 결과와 민감도 분석과 함께 보고한다. 구간은 일치에 대해서는 컨트랙트를, 비용에 대해서는 대상을 2,000번 다시 뽑는 부트스트랩의 백분위 한계로 구한다 [64].

7.7 기술 통계

RQ1에 대해서는 관측 기간 안의 컨트랙트 호출을 포스트컨디션 모드별로 나눈 비율, 발신자에 대한 승인 포스트컨디션이 하나 이상 있는 비율, 선언한 한도와 실제 유출량의 비, Clarity 버전별로 `as-contract?`, `restrict-assets?`, `with-all-assets-unsafe`, 또는 외부 호출을 감싼 옛 `as-contract`를 쓰는 컨트랙트의 비율, `ExecutorDAO` 형태의 컨트랙트와 커진 확장 수, `trait` 타입 인자를 넘기는 호출의 비율을 보고한다. RQ4에 대해서는 쓰인 근거가 없는 컨트랙트의 비율, 묶음 크기의 분포, 가장 큰 묶음들을 보고한다. 또 자주 나오는 함수 이름과 `trait` 이름에서 뽑은 질의 50개에 대해, 묶음으로 모을 때와 모으지 않을 때 상위 10개 결과 가운데 자기 묶음의 대표가 아닌 것의 비율과, 대표가 그 묶음에서 가장 먼저 배포된 것인 경우가 얼마나 되는지 보고한다. 이와 함께 관측 기간 안에서 활동한 지갑이 낸 평균 수수료와 c_{call} 의 비를 보고한다. 이 비는 5.4절에서 수수료 가중치가 지갑 팜의 비용을 몇 배로 높일지 예상한 값이다. 방법마다 그래프 크기와 실행 시간도 보고한다.

7.8 cpm 점검

Clarinet devnet에서 세 경우를 점검한다. SIP-010 토큰 패키지와 그에 의존하는 풀을 `cpm add`로 추가하면 프로젝트 테스트가 통과하는 요구 사항이 만들어지는가, 템플릿에서 `cpm create`로 만든 컨트랙트가 배포되는가, 가짜 API가 소스를 바꾼 컨트랙트를 `cpm verify`가 찾아내는가이다.

8 논의와 한계

포스트컨디션을 쓰는 주체. 사용자가 포스트컨디션을 직접 쓰는 일은 드물다. 애플리케이션의 프런트엔드가 만들고, 지갑이 승인받으려고 보여 준다. ERC-20 승인도 같은 방식으로, 흔히 무제한으로 만들어지므로, 의도의 증거로서 포스트컨디션이 `EndorseRank`가 쓴 `allowance`보다 약하지는 않다. 악의적인 프런트엔드가 한도를 느슨하게 잡을 수는 있지만, PC-`EndorseRank`는 승인을 실제 유출량으로 평가하므로 느슨한 한도는 아무것도 보태지 않는다. `Allow` 모드가 흔한 것으로 드러나면(RQ1) 신호의 상당 부분이 포괄 승인이 된다. 포괄 승인을 뺀 민감도 분석이 순위가 그것에 얼마나 기대는지 보여 준다.

신원 대신 수수료. 지갑을 수수료로 가중하면 순위는 활발한 지갑이 쓰는 컨트랙트에 유리해지고, 많이 쓰는 사용자를 가끔 쓰는 사용자보다 더 무겁게 센다. 그 대안인 지갑당 한 표는 시빌 지갑이 악용하는 규칙이므로(명제 1) 이 점을 받아들인다. 약점은 두 가지가 남는다. Stacks 채굴자는 수수료를 받으므로 수수료를 자기 자신에게 낼 수 있다. 상한 $f_{\max}$ 는 그런 트랜잭션 하나가 살 수 있는 양을 제한하지만 유인을 없애지는 못한다. 또 Stacks의 수수료는 낮아서, 사용자가 적은 범주에서는 공격 비용이 균등 재시작일 때의 몇 배가 되더라도 절대 금액은 작을 수 있다. 다른 가중치인 자본-일은 같은 순간에 여러 지갑에서 재사용할 수 없지만 부유한 지갑에 더 유리하다.

가격. 유출량에 값을 매기려면 가격이 필요하다. 가격은 몫 배분 $s_u(c)$ 에만 들어가고 가중치 $\omega(u)$ 에는 들어가지 않는다. 따라서 조작된 가격은 그 토큰을 보낸 지갑들의 가중치를, 지갑들이 그 토큰을 보낸 컨트랙트 쪽으로 옮길 수 있을 뿐이다. 유동성이 없는 토큰은 가격이 없으므로 몫 배분에 영향을 주지 못한다. T_0 이 미리 정해져 있으므로 T_0 에 맞춘 가격 펌핑이 가장 뻥한 공격이며, 30일 중앙값은 그 펌핑을 오래 유지하는 데 큰 비용이 들게 한다. ERC-20 연구의 관례인 원래 토큰 단위를 쓴 절제 분석이 가격이 얼마나 중요한지 보여 준다.

실행 시점 바인딩. 실행 시점 trait 바인딩을 빼면 인기 있는 라우터가 증폭기로 쓰이는 것을 막을 수 있다. 그 대신 라우터를 통해서만 거래되는 토큰은 그 토큰을 직접 승인한 사용자와, 코드에서 그 토큰을 이름으로 부르는 풀에서만 점수를 받는다. 토큰은 가장 많이 검색되는 컨트랙트에 속하므로, 평가에서는 바인딩을 더한 절제 분석을 보고한다.

정적 추출. 파서는 리터럴, 상대, 상수 대상을 푼다. 실행 중에 데이터 변수나 맵에 저장된 principal은 풀지 못하고, ExecutorDAO 패턴 외의 허용 목록은 읽지 않는다. 이런 참조는 빠진 간선이 되어, 그 간선이 가리켰을 컨트랙트의 점수를 낮춘다. 거버넌스 트랜잭션에서 상태 기반 참조를 뽑는 일은 후속 과제이다.

클론 묶음. 정규화는 코드는 같지만 수수료율처럼 중요한 매개변수가 다른 컨트랙트를 하나로 합칠 수 있고, 함수 이름을 바꾸고 형식을 고친 복제본은 찾지 못한다. 가장 많이 쓰인 구성원을 대표로 고르면 인기 있는 복제본이 원저자보다 위에 올 수 있다. 그래서 레지스트리는 묶음마다 배포 이력을 보여 주고 가장 먼저 배포된 구성원을 표시한다.

데이터 접근과 재현성. 호스팅된 Stacks Blockchain API는 요청 빈도를 제한하며 일부 연구 환경에서는 막혀 있다. 그래서 평가는 직접 운영하는 노드와 API를 쓰고, 추출 스크립트

와 매니페스트를 공개한다. 트랜잭션 수준 데이터에는 지갑 주소가 들어 있다. 공개된 정보이지만 결과는 집계로만 보고하고, 지갑에는 순위를 매기지도 표시하지도 않으며, 레지스트리는 사용자 목록이 아니라 개수만 보여 준다.

새 프로젝트에 대한 불이익. 사용량에 기반한 순위는 새 컨트랙트를 늦게 알아본다. 레지스트리는 쓰이지 않은 컨트랙트도 목록에 올리고 배포일 순으로 정렬한 보기를 제공하며, 점수가 낮다는 이유로 컨트랙트에 부정적인 표시를 붙이지 않는다.

ERC-20 연구와의 관계. 5절은 그 연구에서 단순 개수가 좋은 성적을 낸 까닭을 하나 제시한다. H2는 PC-EndorseRank가 채택과의 일치에서 기준선보다 나쁘지 않기만을 요구한다. PC-EndorseRank가 노리는 이득은 조작 비용을 높이는 것이며, 이것은 H1이 검정한다.

9 결론

Clarity 컨트랙트는 소스로 공개되므로 체인 데이터만으로 레지스트리를 만들 수 있다. 다만 가짜 인기에 버티는 순위가 있어야 한다. Stacks에는 allowance가 없으므로 EndorseRank를 그대로 옮길 수는 없다. allowance 관계는 두 부분으로 나뉜다. 포스트컨디션은 지갑이 어떤 컨트랙트에 자기 자산을 옮기게 했는지를 기록하고, 코드 안의 위임 참조와 의존 참조는 컨트랙트가 어떤 컨트랙트에 기대는지를 기록한다. AWP는 Stacks의 전송에 그대로 적용할 수 있다.

명제들이 가리키는 변화는 하나이다. 지갑을 그래프에서 빼서 재시작 분포를 통해서만 작용하게 하는 것이다. 균등 재시작에서는 컨트랙트 점수 가운데 지갑에서 오는 부분이 지갑 수가 되고, 의존성 PageRank는 대상 주위에 배포한 컨트랙트 수에 따라 커진다. 둘 다 부풀리기 쉽다. 수수료로 가중한 재시작에서는 예산을 여러 지갑에 나눠도 얻는 것이 없고, 어떤 지갑도 승인하지 않고 바깥의 어떤 컨트랙트도 참조하지 않는 컨트랙트 묶음은 점수를 받지 못한다. 합성 예제는 꾸며 낸 그래프에서 이 효과들을 보여 준다. 이 순위 위에 만든 레지스트리는 컨트랙트를 참조하는 일과 복제하는 일을 구분하고, 스코프와 버전을 배포자 주소에 묶고, 클론을 해시로 모으고, 코드 해시를 체인에서 확인한다.

조작 비용이 높아지는 효과가 Stacks 메인넷에서도 성립하는지, 그리고 그 대가로 이후 채택과의 일치를 잃지 않는지는 7절의 평가가 검정한다. 두 결과 모두 방향과 관계없이 보고한다.

참고문헌

- [1] Stacks Improvement Proposals. (n.d.). *SIP-002: Smart contract language*. <https://github.com/stacksgov/sips/tree/main/sips/sip-002>
- [2] Hiro Systems. (2024, August 23). *There's nowhere to hide with Clarity* [Blog post]. <https://www.hiro.so/blog/theres-nowhere-to-hide-with-clarity>
- [3] Hiro Systems. (n.d.). *Stacks Blockchain API reference*. <https://docs.hiro.so/apis/stacks-blockchain-api>
- [4] boomcrypto. (n.d.). *clarity-deployed-contracts* [GitHub repository]. <https://github.com/boomcrypto/clarity-deployed-contracts>
- [5] Stacks Documentation. (n.d.). *Clarinet: Contract deployment and requirements*. <https://docs.stacks.co/clarinet/contract-deployment>
- [6] The Go Authors. (n.d.). *pkgite: internal/postgres/search.go* [Source code]. <https://github.com/golang/pkgite/blob/master/internal/postgres/search.go>
- [7] Sonatype. (2024, April). *Devs flood npm with 15,000 packages to reward themselves with tea 'tokens'* [Blog post]. <https://www.sonatype.com/blog/devs-flood-npm-with-10000-packages-to-reward-themselves-with-tea-tokens>
- [8] Tea Inc. (2022, May 19). *Tea releases whitepaper outlining decentralized protocol for rewarding open source developers* [Press release]. Business Wire. <https://www.businesswire.com/news/home/20220519005165/en/>
- [9] Amazon Web Services. (2025, November 13). *Amazon Inspector detects over 150,000 malicious packages linked to token farming campaign*. AWS Security Blog. <https://aws.amazon.com/blogs/security/amazon-inspector-detects-over-150000-malicious-packages-linked-to-token-farming-campaign/>
- [10] Do, T., Do, D., & Nguyen, L. (2023). On-chain reputation ranking by Adaptive Weighted PageRank. In *2023 RIVF International Conference on Computing and Communication Technologies (RIVF)* (pp. 318–323). IEEE. doi:10.1109/RIVF60135.2023.10471809
- [11] Kwon, T. (2026). *Integrating ERC-20 allowance edges into PageRank for enhanced on-chain wallet reputation scoring* [Doctoral thesis in preparation]. University of South Africa.
- [12] Stacks Improvement Proposals. (n.d.). *SIP-010: Standard trait definition for fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-010>
- [13] Stacks Improvement Proposals. (n.d.). *SIP-005: Blocks, transactions, and accounts*. <https://github.com/stacksgov/sips/tree/main/sips/sip-005>
- [14] Stacks Improvement Proposals. (2025). *SIP-033: Clarity 4*. <https://github.com/stacksgov/sips/tree/main/sips/sip-033>
- [15] Cheng, A., & Friedman, E. (2005). Sybilproof reputation mechanisms. In *Proceedings of the 2005 ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON '05)* (pp. 128–132). ACM.

- [16] Gyöngyi, Z., & Garcia-Molina, H. (2005). Link spam alliances. In *Proceedings of the 31st International Conference on Very Large Data Bases* (pp. 517–528). VLDB Endowment.
- [17] Gyöngyi, Z., Garcia-Molina, H., & Pedersen, J. (2004). Combating web spam with TrustRank. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases* (pp. 576–587). VLDB Endowment.
- [18] Cao, Q., Sirivianos, M., Yang, X., & Pregueiro, T. (2012). Aiding the detection of fake accounts in large scale social online services. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* (pp. 197–210). USENIX Association.
- [19] Ali, M., et al. (n.d.). *Stacks: A Bitcoin layer for smart contracts* [Whitepaper]. <https://stacks-network.github.io/stacks/stacks.pdf>
- [20] Stacks Improvement Proposals. (2026). *SIP-039: Clarity 5: Fixing known issues in Clarity*. <https://github.com/stacksgov/sips/tree/main/sips/sip-039>
- [21] Stacks Improvement Proposals. (n.d.). *SIP-009: Standard trait definition for non-fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-009>
- [22] Stacks Improvement Proposals. (n.d.). *SIP-013: Standard trait definition for semi-fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-013>
- [23] Hiro Systems. (n.d.). *Post-conditions* [Stacks.js documentation]. <https://docs.hiro.so/reference/stacks.js/post-conditions>
- [24] Stacks Improvement Proposals. (2026). *SIP-040: Improved post-conditions*. <https://github.com/stacksgov/sips/tree/main/sips/sip-040>
- [25] Stacks Improvement Proposals. (2026). *SIP-044: Clarity 6, staking and PoX post-conditions, and removal of the cost-voting contract*. <https://github.com/stacksgov/sips/tree/main/sips/sip-044>
- [26] Stacks Documentation. (n.d.). *ExecutorDAO framework*. <https://docs.stacks.co/cookbook/clarity/example-contracts/executordao-framework>
- [27] Kikas, R., Gousios, G., Dumas, M., & Pfahl, D. (2017). Structure and evolution of package dependency networks. In *Proceedings of the 14th International Conference on Mining Software Repositories* (pp. 102–112). IEEE.
- [28] Decan, A., Mens, T., & Grosjean, P. (2019). An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1), 381–416.
- [29] Mujahid, S., Costa, D. E., Abdalkareem, R., Shihab, E., Saied, M. A., & Adams, B. (2021). Towards using package centrality trend to identify packages in decline. *IEEE Transactions on Engineering Management*. doi:10.1109/TEM.2021.3122012
- [30] Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber’s knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)* (pp. 23–43). Springer.

- [31] Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). SoK: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy* (pp. 1509–1526). IEEE.
- [32] Vu, D.-L., Pashchenko, I., Massacci, F., Plate, H., & Sabetta, A. (2020). Typosquatting and com-bosquatting attacks on the Python ecosystem. In *2020 IEEE European Symposium on Security and Privacy Workshops* (pp. 509–514). IEEE.
- [33] Zimmermann, M., Staicu, C.-A., Tenny, C., & Pradel, M. (2019). Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium* (pp. 995–1010). USENIX Association.
- [34] Page, L., Brin, S., Motwani, R., & Winograd, T. (1998). *The PageRank citation ranking: Bringing order to the web* (Technical Report). Stanford Digital Library Technologies Project.
- [35] Langville, A. N., & Meyer, C. D. (2006). *Google’s PageRank and beyond: The science of search engine rankings*. Princeton University Press.
- [36] Haveliwala, T. H. (2003). Topic-sensitive PageRank: A context-sensitive ranking algorithm for web search. *IEEE Transactions on Knowledge and Data Engineering*, 15(4), 784–796. doi:10.1109/TKDE.2003.1208999
- [37] Jeh, G., & Widom, J. (2003). Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web* (pp. 271–279). ACM. doi:10.1145/775152.775191
- [38] Kamvar, S. D., Schlosser, M. T., & Garcia-Molina, H. (2003). The EigenTrust algorithm for reputation management in P2P networks. In *Proceedings of the 12th International Conference on World Wide Web* (pp. 640–651). ACM. doi:10.1145/775152.775242
- [39] Douceur, J. R. (2002). The Sybil attack. In *Peer-to-Peer Systems* (pp. 251–260). Springer. doi:10.1007/3-540-45748-8_24
- [40] Yu, H., Kaminsky, M., Gibbons, P. B., & Flaxman, A. (2006). SybilGuard: Defending against Sybil attacks via social networks. In *Proceedings of the 2006 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM ’06)* (pp. 267–278). ACM.
- [41] Bianchini, M., Gori, M., & Scarselli, F. (2005). Inside PageRank. *ACM Transactions on Internet Technology*, 5(1), 92–128.
- [42] Andersen, R., Borgs, C., Chayes, J., Hopcroft, J., Jain, K., Mirrokni, V., & Teng, S.-H. (2008). Robust PageRank and locally computable spam detection features. In *Proceedings of the 4th International Workshop on Adversarial Information Retrieval on the Web* (pp. 69–76). ACM. doi:10.1145/1451983.1452000
- [43] Do, H., & Do, T. (2023). PageRank and HodgeRank on Ethereum transactions: A measure for social credit. *International Journal of Software Innovation*, 11(1), 1–13. doi:10.4018/IJSI.315737
- [44] Lin, D., Wu, J., Fu, Q., Zheng, Z., & Chen, T. (2025). RiskProp: Account risk rating on Ethereum via de-anonymous score and network propagation. *IEEE Transactions on Dependable and Secure Computing*, 22(3), 2132–2145. doi:10.1109/TDSC.2024.3475825
- [45] Chen, T., et al. (2018). Understanding Ethereum via graph analysis. In *IEEE INFOCOM 2018: IEEE Conference on Computer Communications*. IEEE.

- [46] Victor, F. (2020). Address clustering heuristics for Ethereum. In *Financial Cryptography and Data Security (FC 2020)* (pp. 617–633). Springer. doi:10.1007/978-3-030-51280-4_33
- [47] Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022). Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2353–2367). ACM.
- [48] Kondo, M., Oliva, G. A., Jiang, Z. M., Hassan, A. E., & Mizuno, O. (2020). Code cloning in smart contracts: A case study on verified contracts from the Ethereum blockchain platform. *Empirical Software Engineering*, 25(6), 4617–4675.
- [49] He, N., Wu, L., Wang, H., Guo, Y., & Jiang, X. (2020). Characterizing code clones in the Ethereum smart contract ecosystem. In *Financial Cryptography and Data Security (FC 2020)* (pp. 654–675). Springer.
- [50] Xia, P., Wang, H., Gao, B., Su, W., Yu, Z., Luo, X., Zhang, C., Xiao, X., & Xu, G. (2021). Trade or trick? Detecting and characterizing scam tokens on Uniswap decentralized exchange. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 5(3), 1–26. doi:10.1145/3491051
- [51] Roy, C. K., Cordy, J. R., & Koschke, R. (2009). Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7), 470–495.
- [52] Broder, A. Z. (1997). On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences 1997* (pp. 21–29). IEEE.
- [53] Stacks. (2026, February 5). *Chainhooks v2 is now generally available* [Blog post]. <https://www.stacks.co/blog/chainhooks-v2-is-now-generally-available>
- [54] CoinFabrik. (n.d.). *STACY: Static analyzer for Clarity* [Software]. <https://pypi.org/project/stacy-analyzer>
- [55] Ethereum Improvement Proposals. (2015). *ERC-20 token standard (EIP-20)*. <https://eips.ethereum.org/EIPS/eip-20>
- [56] Ethereum Improvement Proposals. (2020). *ERC-20 permit extension (EIP-2612)*. <https://eips.ethereum.org/EIPS/eip-2612>
- [57] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [58] Stacks Documentation. (n.d.). *How BNSv2 works*. <https://docs.stacks.co/learn/network-fundamentals/bitcoin-name-system/architecture>
- [59] Stacks Improvement Proposals. (n.d.). *SIP-018: Signed structured data*. <https://github.com/stacksgov/sips/tree/main/sips/sip-018>
- [60] DefiLlama. (n.d.). *Stacks: Total value locked by protocol*. <https://defillama.com/chain/Stacks>
- [61] Kendall, M. G. (1938). A new measure of rank correlation. *Biometrika*, 30(1/2), 81–93. doi:10.2307/2332226
- [62] Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1), 72–101. doi:10.2307/1412159

- [63] Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- [64] Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26. doi:10.1214/aos/1176344552

A 추출 규칙

A.1 포스트컨디션

표 6에서는 Stacks Blockchain API가 돌려주는 트랜잭션의 포스트컨디션 필드를 4.4절의 규칙에 대응시킨다 [3]. 필드 이름은 이 글을 쓰는 시점의 API 스키마를 따랐다. 평가에서는 설정 파일에 API 버전 하나를 고정하고, Originator 모드와 SIP-040의 *may send* 조건이 어떻게 표현되는지 그 버전에서 확인한다.

표 6: 포스트컨디션 필드와 추출에서의 쓰임

필드	쓰임
<code>tx_status</code>	승인은 성공한 트랜잭션에서만 얻는다. 수수료는 실패한 것을 포함해 지갑이 발신한 모든 트랜잭션에 대해 센다.
<code>tx_type</code> , <code>contract_call.contract_id</code>	컨트랙트 호출만 쓴다. 여기 적힌 컨트랙트가 진입 컨트랙트 $c(\tau)$ 이다.
<code>sender_address</code> , <code>sponsored</code>	발신자 $o(\tau)$. 수수료는 대납 여부와 관계없이 발신자의 것으로 센다.
<code>fee_rate</code>	수수료 $f(\tau)$. $f_{\max}$ 에서 자른다.
<code>post_condition_mode</code>	Deny와 Originator: 포스트컨디션이 다루는 발신자의 유출만 명시적 승인으로 센다. Allow: 포스트컨디션이 다루지 않는 발신자의 유출을 포괄 승인으로 세고 표시해 둔다.
<code>post_conditions[].principal.type_id</code>	<code>principal_origin</code> , 또는 발신자 주소와 같은 <code>principal_standard</code> : 승인이 될 수 있다. 컨트랙트 <code>principal</code> : 승인이 아니다.
<code>post_conditions[].type</code>	<code>stx</code> , <code>fungible</code> , <code>non_fungible</code> . <code>asset</code> 과 함께 자산 a 를 식별한다.
<code>post_conditions[].condition_code</code>	상한(<code>sent_equal_to</code> , <code>sent_less_than</code> , <code>sent_less_than_or_equal_to</code>)과 NFT의 <code>sent</code> 또는 <i>may send</i> : 명시적 승인. 발신자에 대한 하한: 포괄 승인. <code>not_sent</code> : 승인 아님.
<code>events[]</code> (자산 전송과 소각)	실제 유출량 $out_a(\tau)$: 발신자가 보내거나 소각한 양.

A.2 Clarity 구문

표 7에 분석기가 읽는 구문과 그 구문이 만드는 간선을 나열했다.

표 7: 컨트랙트 A 의 Clarity 구문과 그 구문이 만드는 간선

A 안의 구문	간선
<code>(contract-call? 'SP...b f ...)</code>	의존 $A \rightarrow b$
<code>(contract-call? .b f ...)</code>	같은 배포자의 b 로 가는 의존 $A \rightarrow b$
<code>(define-constant T 'SP...b)와</code> <code>(contract-call? T f ...)</code> (Clarity 5)	의존 $A \rightarrow b$
<code>(impl-trait 'SP...d.t), (use-trait x 'SP...d.t)</code>	trait t 를 정의한 컨트랙트 d 로 가는 의존 $A \rightarrow d$
<code>as-contract?, as-contract,</code> <code>restrict-assets?</code> 안에서 대상을 알아낸 b 호출	위임 $A \rightarrow b$
ExecutorDAO 형태인 A 에서 b 를 컨 가장 최근의 <code>set-extension</code> 호출	위임 $A \rightarrow b$
x 가 trait 타입 인자인 <code>(contract-call?</code> <code>x f ...)</code>	간선 없음. 동적 디스패치로 기록한다.
<code>with-all-assets-unsafe;</code> 외부 호출을 감싼 <code>as-contract;</code> <code>as-contract?</code> 안의 trait 타입 호출	위 행들에서 생기는 간선과 별도로 신호로 보여 준다 (표 4).

B 합성 예제

5.7절의 예제는 난수 시드를 고정한 `scripts/toy_sybil_example.py`로 만든다. 정직한 컨트랙트 14개는 SIP-010 trait 하나, 그 trait를 구현하는 토큰 여섯 개, 수학 라이브러리 하나, trait와 라이브러리를 쓰고 `as-contract?` 안에서 두 토큰을 내주는 풀 세 개, 세 풀을 호출하는 라우터, 두 토큰을 내주고 라이브러리를 쓰는 볼트, 볼트와 라우터를 확장으로 컨 DAO로 이루어진다. 정직한 지갑 300개는 저마다 진입 컨트랙트(라우터, 볼트, DAO, 토큰 중) 한 개에서 세 개를 로그정규분포에서 뽑은 값으로 승인하고, $1 + \text{Poisson}(8)$ 개 트랜잭션의 수수료를 낸다. 트랜잭션 하나의 수수료는 최소 수수료에 중앙값이 3인 로그정규분포의 배수를 곱한 값이다. 라우터를 쓰는 지갑은 라우터를 한 번에서 다섯 번 호출하며, 호출마다 풀 하나의 두 토큰을 바인딩한다. 배포 한 번에는 최소 수수료의 10배가 든다. 순위는 공격자의 컨트랙트를 포함해 점수가 대상보다 엄격히 높은 컨트랙트만 세므로, 동점은 대상에게 유리하게 처리되고 표 2의 비용은 실제보다 낮게 나올 수는 있어도 높게 나오지는 않는다. PageRank는 5절의 설정을 쓰고, 수수료 상한 $f_{\max}$ 는 땀다. 실행 시점 바인딩을 더한 절제 분석에서는 라우터의 바인딩 간선 전체에 라우터의 정적 참조와 같은 가중치를 주고, 그 가중치를 토큰이 바인딩된 횟수에 비례해 나눈다. 표 8에 공격 크기별 대상의 순위를 적었다.

표 8: 합성 예제에서 공격자 토큰이 전체 컨트랙트 가운데 차지하는 순위(1이 맨 위). k 는 팜 컨트랙트(A1), 시빌 지갑(A2), 라우터 호출(A4)의 수이다. 열은 표 2의 점수 규칙으로, 차례로 서로 다른 호출자 수, 의존성 PageRank, 균등 재시작으로 Stacks에 옮긴 EndorseRank, PC-EndorseRank, 실행 시점 trait 바인딩을 더한 PC-EndorseRank이다.

공격	k	호출자 수	의존성 PR	ER(균등)	PC-ER	PC-ER+바인딩
A1	0	10	14	15	15	15
A1	10	10	2	15	15	15
A1	100	10	1	2	15	15
A1	1,000	10	1	2	15	15
A2	0	10	14	15	15	15
A2	10	10	14	15	15	15
A2	100	3	14	2	15	15
A2	1,000	1	14	2	7	8
A4	0	10	14	15	15	15
A4	10	10	14	15	15	15
A4	100	10	14	15	15	15
A4	1,000	10	14	15	15	9